

ГЛАВА 9

ПОСЛЕДОВАТЕЛЬНЫЕ КОНТЕЙНЕРЫ

В этой главе...

9.1. Обзор последовательных контейнеров	420
9.2. Обзор библиотечных контейнеров	423
9.3. Операции с последовательными контейнерами	439
9.4. Как увеличивается размер вектора	457
9.5. Дополнительные операции со строками	462
9.6. Адаптеры контейнеров	473
Резюме	477
Термины	477

Эта глава подробно останавливается на материале главы 3 и завершает обсуждение последовательных контейнеров стандартной библиотеки. Порядок элементов в последовательном контейнере соответствует порядку их добавления в контейнер. В библиотеке определено также несколько ассоциативных контейнеров, позиция элементов которых зависит от ключа, ассоциируемого с каждым элементом. Операции, специфические для ассоциативных контейнеров, рассматриваются в главе 11.

Классы контейнеров имеют общий интерфейс, который каждый из контейнеров дополняет собственным способом. Общий интерфейс упрощает изучение библиотечных классов; то, что стало известно о контейнере одного вида, относится к контейнеру другого. Однако каждый вид контейнеров обладает индивидуальной эффективностью и функциональными возможностями.

Контейнер (container) содержит коллекцию объектов определенного типа. *Последовательные контейнеры* (sequential container) позволяют контролировать порядок, в котором хранятся элементы и предоставляется доступ к ним. Этот порядок не зависит от значений элементов, он соответствует позиции, в которую помещаются элементы контейнера. В отличие от них, ассоциативные контейнеры (упорядоченные и неупорядоченные) хранят свои элементы на основании значения ключа, как будет описано в главе 11.

Библиотека предоставляет также три контейнерных адаптера, каждый из которых адаптирует определенный тип контейнера, определяя иной интерфейс к функциям контейнера. Адаптеры рассматриваются в конце этой главы.



Эта глава основана на материале разделов 3.2–3.4. Здесь подразумевается, что читатель знаком с их материалом.



9.1. Обзор последовательных контейнеров

Все последовательные контейнеры, перечисленные в табл. 9.1, предоставляют быстрый последовательный доступ к своим элементам. Однако эти контейнеры обладают разной производительностью и возможностями, включая следующие:

- цена добавления и удаления элементов из контейнера;
- цена непоследовательного доступа к элементам контейнера.

Таблица 9.1. Типы последовательных контейнеров

<code>vector</code>	Массив переменного размера (вектор). Обеспечивает быстрый произвольный доступ. Вставка и удаление элементов, кроме как в конец, могут быть продолжительными
<code>deque</code>	Двухсторонняя очередь. Обеспечивает быстрый произвольный доступ. Быстрая вставка и удаление в начало и конец
<code>list</code>	Двухсвязный список. Обеспечивает только двунаправленный последовательный доступ. Быстрая вставка и удаление в любую позицию
<code>forward_list</code>	Односвязный список. Обеспечивает только последовательный доступ в одном направлении. Быстрая вставка и удаление в любую позицию
<code>array</code>	Массив фиксированного размера. Обеспечивает быстрый произвольный доступ. Не позволяет добавлять или удалять элементы
<code>string</code>	Специализированный контейнер, подобный вектору, содержащий символы. Быстрый произвольный доступ. Быстрая вставка и удаление в конец

За исключением контейнера `array`, являющегося контейнером фиксированного размера, контейнеры обеспечивают эффективное, гибкое управление памятью, позволяя добавлять и удалять элементы, увеличивая и сокращая размер контейнера. Стратегии, используемые контейнерами для хранения своих элементов, имеют незначительное, а иногда существенное влияние на эффективность операций с ними. В некоторых случаях эти стратегии влияют также на то, поддерживает ли некий контейнер определенную операцию.

Например, контейнеры `string` и `vector` содержат свои элементы в соседних областях памяти. Поскольку элементы расположены последовательно, их адрес по индексу вычисляется очень быстро. Но добавление или удаление элемента в середину такого контейнера занимает много времени: для обеспечения последовательности все элементы после удаления или вставки придется переместить. Кроме того, добавление элемента может иногда потребовать резервирования дополнительного места для хранения. В этом случае каждый элемент следует переместить в новое место.

Контейнеры `list` и `forward_list` разработаны так, чтобы быстро добавлять и удалять элементы в любой позиции контейнера. Однако эти типы не поддерживают произвольный доступ к элементам: обратиться к элементу можно, только перебрав контейнер. Кроме того, дополнительные затраты памяти этих контейнеров зачастую являются существенными по сравнению с контейнерами `vector`, `deque` и `array`.

Контейнер `deque` — это более сложная структура данных. Как и контейнеры `string`, `vector` и `deque`, они обеспечивают быстрый произвольный доступ. Как и у контейнеров `string` и `vector`, добавление или удаление элементов в середину контейнера `deque` — потенциально дорогая операция. Однако добавление и удаление элементов в оба конца контейнера `deque` являются быстрой операцией, сопоставимой с таковыми у контейнеров `list` и `forward_list`.

**C++
11**

Типы `forward_list` и `array` были добавлены согласно новому стандарту. Класс `array` — более безопасная и легкая в употреблении альтернатива встроенным массивам. Как и встроенные массивы, объект библиотечного класса `array` имеет фиксированный размер. В результате он не поддерживает операции по добавлению и удалению элементов или изменению размеров контейнера. Класс `forward_list` предназначен для замены наилучших самодельных односвязных списков. Следовательно, у него нет функции `size()`, поскольку сохранение или вычисление его размера повлекло бы дополнительные затраты по сравнению с самодельным списком. Для других контейнеров функция `size()` гарантированно будет выполняться быстро с постоянной скоростью.



По причинам, изложенным в разделе 13.6 (стр. 672), новые библиотечные контейнеры работают существенно быстрее, чем в предыдущих версиях. Библиотечные контейнеры почти наверняка работают так же (если не лучше), чем даже наиболее тщательно разработанные альтернативы. Современные программы C++ должны использовать библиотечные контейнеры, а не множество примитивных структур, подобных массивам.

Решение о том, какой последовательный контейнер использовать



Как правило, если нет серьезных оснований предпочесть другой контейнер, лучше использовать контейнер `vector`.

Ниже приведено несколько эмпирических правил по выбору используемого контейнера.

- Если нет причин использовать другой контейнер, используйте вектор.
- Если имеется много небольших элементов и дополнительных затрат, не используйте контейнер `list` или `forward_list`.
- Если нужен произвольный доступ к элементам, используйте контейнер `vector` или `deque`.
- Если необходима вставка или удаление элементов в середину, используйте контейнер `list` или `forward_list`.
- Если необходима вставка или удаление элементов в начало или в конец, но не в середину, используйте контейнер `deque`.
- Если вставка элементов в середину контейнера нужна только во время чтения ввода, а впоследствии нужен произвольный доступ к элементам, то:
 - сначала решите, необходимо ли фактически добавлять элементы в середину контейнера. Зачастую проще добавлять элементы в `vector`, а затем использовать библиотечную функцию `sort()` (рассматриваемую в разделе 10.2.3 (стр. 492)) для переупорядочивания контейнера по завершении ввода;
 - если вставка в середину необходима, рассмотрите возможность использования контейнера `list` на фазе ввода, а по его завершении — копирования списка в вектор.

Но что если нужен и произвольный доступ, и вставка (удаление) элементов в середину контейнера? Решение будет зависеть от отношения цены доступа к элементам в контейнере `list` и `forward_list` цены вставки (удаления) элементов в контейнер `vector` или `deque`. Как правило, выбор типа контейнера определит преобладающая операция приложения (произвольный доступ или вставка и удаление). В таких случаях, вероятно, потребуется проверка производительности приложения с использованием контейнеров обоих типов.



Если вы не уверены, какой контейнер использовать, напишите свой код так, чтобы он использовал только те операции, которые совпадают у вектора и списка: используйте итераторы, а не индексы, и избегайте произвольного доступа к элементам. Так будет удобней заменить вектор на список при необходимости.

Упражнения раздела 9.1

Упражнение 9.1. Какой из контейнеров (`vector`, `deque` или `list`) лучше подходит для приведенных ниже задач? Объясните, почему. Если нельзя отдать предпочтение тому или иному контейнеру, объясните, почему?

- Чтение известного заранее количества слов и вставка их в контейнер в алфавитном порядке по мере ввода. В следующей главе будет показано, что ассоциативные контейнеры лучше подходят для этой задачи.
- Чтение неизвестного заранее количества слов. Новые слова всегда добавляются в конец. Следующее значение извлекается из начала.
- Чтение неизвестного заранее количества целых чисел из файла. Числа сортируются, а затем выводятся на стандартное устройство вывода.



9.2. Обзор библиотечных контейнеров

Возможные операции с контейнерами составляют своего рода иерархию.

- Некоторые функциональные возможности (табл. 9.2, стр. 423) поддерживаются контейнерами всех типов.
- Другие операции являются специфическими только для последовательных (табл. 9.3, стр. 431), ассоциативных (табл. 11.7, стр. 558) или неупорядоченных (табл. 11.8, стр. 568) контейнеров.
- Остальные являются общими лишь для небольшого подмножества контейнеров.

Таблица 9.2. Средства контейнеров

Псевдонимы типов	
<code>iterator</code>	Тип итератора для контейнера данного типа
<code>const_iterator</code>	Тип итератора, позволяющий читать, но не изменять значение элемента
<code>size_type</code>	Целочисленный беззнаковый тип, размер которого достаточно велик, чтобы содержать значение размера наибольшего возможного контейнера данного типа
<code>difference_type</code>	Целочисленный знаковый тип, размер которого достаточно велик, чтобы содержать значение разницы между двумя итераторами
<code>value_type</code>	Тип элемента
<code>Reference</code>	Тип l-значения элемента; то же, что и <code>value_type&</code>
<code>const_reference</code>	Тип константного l-значения элемента; аналог <code>const value_type&</code>

Конструкторы	
<code>c c;</code>	Стандартный конструктор, создающий пустой контейнер
<code>c c1(c2);</code>	Создает контейнер <code>c1</code> как копию контейнера <code>c2</code>
<code>c c(b, e);</code>	Копирует элементы из диапазона, обозначенного итераторами <code>b</code> и <code>e</code> (<i>недопустимо для массива</i>)
<code>c c{a,b,c...};</code>	Списочная инициализация контейнера <code>c</code>
Присвоение и замена	
<code>c1 = c2</code>	Заменяет элементы контейнера <code>c1</code> элементами контейнера <code>c2</code>
<code>c1 = {a,b,c...}</code>	Заменяет элементы контейнера <code>c1</code> элементами списка (<i>недопустимо для массива</i>)
<code>a.swap(b)</code>	Меняет местами элементы контейнеров <code>a</code> и <code>b</code>
<code>swap(a, b)</code>	Эквивалент <code>a.swap(b)</code>
Размер	
<code>c.size()</code>	Возвращает количество элементов контейнера <code>c</code> (<i>недопустимо для контейнера <code>forward_list</code></i>)
<code>c.max_size()</code>	Возвращает максимально возможное количество элементов контейнера <code>c</code>
<code>c.empty()</code>	Возвращает логическое значение <code>false</code> , если контейнер <code>c</code> пуст. В противном случае возвращает значение <code>true</code>
Добавление/удаление элементов (недопустимо для массива)	
Примечание: интерфейс этих функций зависит от типа контейнера	
<code>c.insert(args)</code>	Копирует элемент(ы), указанный параметром <code>args</code> , в контейнер <code>c</code>
<code>c.emplace(inits)</code>	Использует параметр <code>inits</code> для создания элемента в контейнере <code>c</code>
<code>c.erase(args)</code>	Удаляет элемент(ы), указанный параметром <code>args</code> , из контейнера <code>c</code>
<code>c.clear()</code>	Удаляет все элементы из контейнера <code>c</code> ; возвращает значение <code>void</code>
Операторы равенства и отношения	
<code>==, !=</code>	Равенство допустимо для контейнеров всех типов
<code><, <=, >, >=</code>	Операторы отношения (<i>недопустимы для неупорядоченных ассоциативных контейнеров</i>)

Окончание табл. 9.2

Получения итераторов	
<code>c.begin(), c.end()</code>	Возвращают итератор на первый и следующий после последнего элемент в контейнере <code>c</code>
<code>c.cbegin(), c.cend()</code>	Возвращают <code>const_iterator</code>
Дополнительные члены реверсивных контейнеров (недопустимы для <code>forward_list</code>)	
<code>reverse_iterator</code>	Итератор, обеспечивающий доступ к элементам в обратном порядке
<code>const_reverse_iterator</code>	Реверсивный итератор, не позволяющий запись в элементы
<code>c.rbegin(), c.rend()</code>	Возвращает итератор на последний и следующий после первого элементы контейнера <code>c</code>
<code>c.crbegin(), c.crend()</code>	Возвращают итератор <code>const_reverse_iterator</code>

В этом разделе рассматриваются аспекты, являющиеся общими для всех контейнеров. Остальная часть этой главы посвящена исключительно последовательным контейнерам; операции, специфические для ассоциативных контейнеров, рассматриваются в главе 11.

Обычно каждый контейнер определяется в файле заголовка, название которого совпадает с именем типа. Таким образом, тип `deque` определен в заголовке `deque`, тип `list` — в заголовке `list` и т.д. Контейнеры — это шаблоны классов (см. раздел 3.3, стр. 141). Подобно векторам, при создании контейнера специфического типа необходимо предоставить дополнительную информацию. Для большинства контейнеров, но не всех, предоставляемой информацией является тип элемента:

```
list<Sales_data> // список, содержащий объекты класса Sales_data
deque<double>    // двухсторонняя очередь переменных типа double
```

Ограничения на типы, которые может содержать контейнер

Типом элемента последовательного контейнера может быть практически любой тип. В частности, типом элемента контейнера может быть другой контейнер. Такие контейнеры определяют точно так же, как любые другие: в угловых скобках указывается тип элемента (которым в данном случае является другой контейнер):

```
vector<vector<string>> lines; // вектор векторов
```

где `lines` — это вектор, элементами которого являются векторы строк.



Устаревшие компиляторы могут потребовать пробела между угловыми скобками, например `vector<vector<string> >`.

Несмотря на то что в контейнере можно хранить практически любой тип, некоторые операции налагают на тип элемента собственные требования. Можно определить контейнер для типа, который не поддерживает определенное операцией требование, но использовать операцию можно, только если тип элемента отвечает требованиям этой операции.

В качестве примера рассмотрим конструктор последовательного контейнера, получающий аргумент размера (см. раздел 3.3.1, стр. 143) и использующий стандартный конструктор типа элемента. У некоторых классов нет стандартного конструктора. Вполне можно определить контейнер, содержащий объекты такого типа, но создать такой контейнер, используя только количество элементов, нельзя:

```
// тип noDefault не имеет стандартного конструктора
vector<noDefault> v1(10, init); // ok: предоставлен инициализатор
                                // элемента
vector<noDefault> v2(10);       // ошибка: необходимо предоставить
                                // инициализатор элемента
```

Поскольку рассматриваются контейнерные операции, следует заметить, что тип элемента накладывает дополнительные ограничения, если таковые вообще имеются, на каждую операцию с контейнером.

Упражнения раздела 9.2

Упражнение 9.2. Определите список (*list*), элементами которого будут двухсторонние очереди целых чисел.



9.2.1. Итераторы

Подобно контейнерам, у итераторов есть общий интерфейс: если итератор поддерживает некую функцию, то аналогичным образом она работает с каждым поддерживающим ее итератором. Например, итераторы всех контейнеров стандартных типов позволяют обращаться к элементу, предоставляя оператор обращения к значению. Аналогично все итераторы библиотечных контейнеров определяют оператор инкремента для перемещения от одного элемента к следующему.

За одним исключением контейнерные итераторы поддерживают все функции, перечисленные в табл. 3.6 (стр. 154). Исключение в том, что итераторы контейнера *forward_list* не поддерживают оператор декремента (*--*). Операторы арифметических действий с итераторами, перечисленными в табл. 3.7 (стр. 159), применимы только к итераторам контейнеров *string*, *vector*, *deque* и *array*. К итераторам контейнеров любых других типов эти операторы неприменимы.

Диапазоны итераторов



Концепция диапазона итераторов фундаментальна для стандартной библиотеки.

Диапазон итераторов (iterator range) обозначается парой итераторов, каждый из которых указывает на элемент или на *следующий элемент после последнего* в том же контейнере. Эти два итератора, обозначающие диапазон элементов контейнера, зачастую называют `begin` и `end` или, что несколько обманчиво, `first` и `last`.

Хоть имя `last` и общепринято, оно немного вводит в заблуждение, поскольку второй итератор никогда не указывает на последний элемент диапазона. Вместо этого он указывает на позицию следующего элемента после последнего. Диапазон включает элемент, обозначенный итератором `first`, и все элементы от него до обозначенного итератором `last`, но не включая его.

Такой диапазон элементов называется *интервал, включающий левый элемент* (left-inclusive interval). Вот стандартная математическая форма записи такого диапазона:

[`begin`, `end`)

Это указывает, что диапазон начинается с элемента, обозначенного итератором `begin`, и заканчивается элементом перед тем, который обозначен итератором `end`. Итераторы `begin` и `end` должны относиться к тому же контейнеру. Итератор `end` может быть равен итератору `begin`, но не должен указывать на элемент перед обозначенным итератором `begin`.

ТРЕБОВАНИЯ К ИТЕРАТОРАМ, ФОРМИРУЮЩИМ ДИАПАЗОН

Два итератора, `begin` и `end`, позволяют задать диапазон при следующих условиях.

- Итераторы относятся к существующим элементам или к следующему элементу за концом того же контейнера.
- Элемент `end` достижимым благодаря последовательному приращению итератора `begin`. Другими словами, итератор `end` не должен предшествовать итератору `begin`.



ВНИМАНИЕ

Компилятор не может сам соблюдать эти требования. Позаботиться об этом придется разработчику.

Смысл использования диапазонов, включающих левый элемент

Библиотека использует диапазоны, включающие левый элемент, потому, что они обладают двумя очень полезными качествами (напомним, что допустимый диапазон обозначают итераторы `begin` и `end`).

- Если итератор `begin` равен итератору `end`, то диапазон пуст.
- Если итератор `begin` не равен итератору `end`, в диапазоне содержится по крайней мере один элемент и итератор `begin` указывает на первый из них.
- Можно осуществлять инкремент итератора `begin` до тех пор, пока он не станет равен итератору `end` (т.е. `begin == end`).

Благодаря этим качествам можно создавать вполне безопасные циклы обработки диапазона элементов, например, такие:

```
while (begin != end) {
    *begin = val; // ok: диапазон не пуст, begin обозначает элемент
    ++begin;      // переместить итератор и получить следующий элемент
}
```

Если итераторы `begin` и `end` задают допустимый диапазон элементов, выполнение условия `begin == end` означает, что диапазон пуст. В данном случае это условие выхода из цикла. Если диапазон не пуст, значит, итератор `begin` указывает на элемент в этом не пустом диапазоне. Вполне очевидно, что в теле цикла `while` можно безопасно обращаться к значению итератора `begin`, поскольку оно гарантировано существует. И наконец, поскольку инкремент итератора `begin` осуществляется в теле цикла, последний гарантированно будет конечным.

Упражнения раздела 9.2.1

Упражнение 9.3. Каким условиям должны удовлетворять итераторы, обозначающие диапазон?

Упражнение 9.4. Напишите функцию, которая получает два итератора вектора `vector<int>` и значение типа `int`. Организуйте поиск этого значения в диапазоне и возвратите логическое значение (тип `bool`), указывающее, что значение найдено.

Упражнение 9.5. Перепишите предыдущую программу так, чтобы она возвращала итератор на найденный элемент. Функция должна учитывать случай, когда элемент не найден.

Упражнение 9.6. Что не так со следующей программой? Как ее можно исправить?

```
list<int> lst1;  
list<int>::iterator iter1 = lst1.begin(),  
iter2 = lst1.end();  
while (iter1 < iter2) /* ... */
```

9.2.2. Типы-члены классов контейнеров

Класс каждого контейнера определяет несколько типов, представленных в табл. 9.2 (стр. 423). Три из них уже использовались: `size_type` (см. раздел 3.2.2 стр. 131), `iterator` и `const_iterator` (см. раздел 3.4.1 стр. 155).

Кроме итераторов уже использовавшихся типов, большинство контейнеров предоставляет *реверсивные итераторы* (*reverse iterator*). Другими словами, реверсивный итератор — это итератор, перебирающий контейнер назад и инвертирующий значение его операторов. Например, оператор `++` возвращает реверсивный итератор к предыдущему элементу. Более подробная информация о реверсивных итераторах приведена в разделе 10.4.3 (стр. 521).

Остальные псевдонимы типов позволяют использовать тип хранящихся в контейнере элементов, даже не зная его конкретно. Если необходим тип элемента, используется тип `value_type` контейнера. Если необходима ссылка на этот тип, используется член `reference` или `const_reference`. Эти связанные с элементами псевдонимы типов весьма полезны в обобщенном программировании, которое рассматривается в главе 16.

Чтобы использовать один из этих типов, следует указать класс, членами которого они являются:

```
// iter имеет тип iterator, определенный классом list<string>  
list<string>::iterator iter;  
// count имеет тип difference_type, определенный классом vector<int>  
vector<int>::difference_type count;
```

В этих объявлениях оператор области видимости (см. раздел 1.2, стр. 33) позволяет указать, что используется тип-член `iterator` класса `list<string>` и тип-член `difference_type`, определенный классом `vector<int>`, соответственно.

Упражнения раздела 9.2.2

Упражнение 9.7. Какой тип следует использовать в качестве индекса для вектора целых чисел?

Упражнение 9.8. Какой тип следует использовать для чтения элементов в списке строк?



9.2.3. Функции-члены `begin()` и `end()`

Функции-члены `begin()` и `end()` (см. раздел 3.4.1, стр. 153) возвращают итераторы на первый и следующий после последнего элементы контейнера соответственно. Эти итераторы, как правило, используют при создании диапазона итераторов, охватывающего все элементы контейнера.

Как показано в табл. 9.2 (стр. 423), есть несколько версий этих функций: имена которых начинаются с буквы `r` возвращают реверсивные итераторы (рассматриваются в разделе 10.4.3 (стр. 521)), а с буквы `c` — возвращают константную версию соответствующего итератора:

```
list<string> a = {"Milton", "Shakespeare", "Austen"};
auto it1 = a.begin(); // list<string>::iterator
auto it2 = a.rbegin(); // list<string>::reverse_iterator
auto it3 = a.cbegin(); // list<string>::const_iterator
auto it4 = a.crbegin(); // list<string>::const_reverse_iterator
```

Функции, имена которых не начинаются с буквы `c`, перегружены. Таким образом, фактически есть две функции-члена `begin()`. Одна является константной (см. раздел 7.1.2, стр. 337) и возвращает тип `const_iterator` контейнера. Вторая не константна и возвращает тип `iterator` контейнера. Аналогично для функций `rbegin()`, `end()` и `rend()`. При вызове такой функции-члена для неконстантного объекта используется версия, возвращающая тип `iterator`. Константная версия итераторов будет получена *только* при вызове этих функций для константного объекта. Подобно указателям и ссылкам на константу, итератор типа `iterator` можно преобразовать в соответствующий итератор типа `const_iterator`, но не наоборот.



Версии этих функций, имена которых не начинаются с буквы `c`, были введены согласно новому стандарту для обеспечения использования ключевого слова `auto` с функциями `begin()` и `end()` (см. раздел 2.5.2, стр. 107). Прежде не было никакого иного выхода, кроме как явно указать необходимый тип итератора:

```
// тип указан явно
list<string>::iterator it5 = a.begin();
list<string>::const_iterator it6 = a.begin();
// iterator или const_iterator в зависимости от типа a
auto it7 = a.begin(); // const_iterator только если a константа
auto it8 = a.cbegin(); // it8 - const_iterator
```

Когда с функциями `begin()` или `end()` используется ключевое слово `auto`, тип возвращаемого итератора зависит от типа контейнера. То, как предполагается использовать итератор, несущественно. Версии `c` позволяют получать итератор типа `const_iterator` независимо от типа контейнера.



Когда доступ на запись не нужен, используйте версии `cbegin()` и `cend()`.

Упражнения раздела 9.2.3

Упражнение 9.9. В чем разница между функциями `begin()` и `cbegin()`?

Упражнение 9.10. Каковы типы следующих четырех объектов?

```
vector<int> v1;
const vector<int> v2;
auto it1 = v1.begin(), it2 = v2.begin();
auto it3 = v1.cbegin(), it4 = v2.cbegin();
```



9.2.4. Определение и инициализация контейнера

Каждый контейнерный тип определяет стандартный конструктор (см. раздел 7.1.4, стр. 343). За исключением контейнера `array` стандартный конструктор создает пустой контейнер определенного типа. Также за исключением контейнера `array` другие конструкторы получают аргументы, которые определяют размер контейнера и исходные значения его элементов.

Инициализация контейнера как копии другого контейнера

Существуют два способа создать новый контейнер как копию другого: можно непосредственно скопировать контейнер или (за исключением контейнера `array`) скопировать только диапазон его элементов, обозначенный парой итераторов.

Таблица 9.3. Определение и инициализация контейнера

<code>C c;</code>	Стандартный конструктор. Если <code>C</code> — массив, то элементы контейнера <code>c</code> инициализируются значением по умолчанию; в противном случае контейнер <code>c</code> пуст
<code>C c1(c2) C c1 = c2</code>	Контейнер <code>c1</code> — копия <code>c2</code> . Контейнеры <code>c1</code> и <code>c2</code> должны быть одинакового типа (т.е. должны иметь тот же тип контейнера и содержать элементы того же типа; у массивов должен также совпадать размер)
<code>C c{a,b,c...}</code> <code>C c = {a,b,c...}</code>	Контейнер <code>c</code> содержит копию элементов из списка инициализации. Тип элементов в списке должен быть совместимым с типом элементов <code>C</code> . В случае массива количество элементов списка не должно превышать размер массива, а все недостающие элементы инициализируются значением по умолчанию (см. раздел 3.3.1, стр. 143)
<code>C c(b, e)</code>	Контейнер <code>c</code> содержит копию элементов из диапазона, обозначенного итераторами <code>b</code> и <code>e</code> . Тип элементов должен быть совместимым с типом элементов <code>C</code> . (Недопустимо для массива.)

Получающие размер конструкторы допустимы только для последовательных контейнеров (исключая массив)

<code>C seq(n)</code>	Контейнер <code>seq</code> содержит <code>n</code> элементов, инициализированных значением по умолчанию; этот конструктор является явным (см. раздел 7.5.4, стр. 383). (Недопустимо для строки.)
<code>C seq(n, t)</code>	Контейнер <code>seq</code> содержит <code>n</code> элементов со значением <code>t</code>

Чтобы создать контейнер как копию другого, их типы контейнеров и элементов должны совпадать. При передаче итераторов идентичность типов контейнеров необязательна. Кроме того, могут отличаться типы элементов нового и исходного контейнеров, если возможно преобразование между ними (см. раздел 4.11 стр. 218):

```
// каждый контейнер имеет три элемента, инициализированных
// предоставленными инициализаторами
list<string> authors = {"Milton", "Shakespeare", "Austen"};
vector<const char*> articles = {"a", "an", "the"};
list<string> list2(authors);    // ok: типы совпадают
deque<string> authList(authors); // ошибка: типы контейнеров не совпадают
vector<string> words(articles); // ошибка: типы элементов не совпадают
// ok: преобразует элементы const char* в string
forward_list<string> words(articles.begin(), articles.end());
```



При копировании содержимого одного контейнера в другой типы контейнеров и их элементов должны точно совпадать.

Конструктор, получающий два итератора, использует их для обозначения диапазона копируемых элементов. Как обычно, итераторы отмечают первый и следующий элемент после последнего копируемого. Размер нового контейнера будет совпадать с количеством элементов в диапазоне. Каждый элемент в новом контейнере инициализируется значением соответствующего элемента в диапазоне.

Поскольку итераторы обозначают диапазон, этот конструктор можно использовать для копирования части последовательности. С учетом того что `it` является итератором, обозначающим элемент в контейнере `authors`, можно написать следующее:

```
// копирует до, но не включая элемент, обозначенный итератором it
deque<string> authList(authors.begin(), it);
```

Списочная инициализация



По новому стандарту контейнер допускает списочную инициализацию (см. раздел 3.3.1, стр. 143):

```
// каждый контейнер имеет три элемента, инициализированных
// предоставленными инициализаторами
list<string> authors = {"Milton", "Shakespeare", "Austen"};
vector<const char*> articles = {"a", "an", "the"};
```

Это определяет значение каждого элемента в контейнере явно. Кроме контейнеров таких типов, как `array`, список инициализации неявно определяет также размер контейнера: у контейнера будет столько элементов, сколько инициализаторов в списке.

Конструкторы последовательных контейнеров, связанные с размером

Кроме конструкторов, общих для последовательных и ассоциативных контейнеров, последовательные контейнеры (кроме массива) можно также инициализировать, указав их размера и (необязательного) инициализирующий элемент. Если его не предоставить, библиотека создает инициализирующее значение сама (см. раздел 3.3.1, стр. 142):

```
vector<int> ivec(10, -1); // десять элементов типа int; значение -1
list<string> svec(10, "hi!"); // десять строк; значение "hi!"
forward_list<int> ivec(10); // десять элементов; значение 0
deque<string> svec(10); // десять элементов; все пустые строки
```

Конструктор, получающий аргумент размера, можно также использовать, если элемент имеет встроенный тип или тип класса, у которого есть стандартный конструктор (см. раздел 9.2, стр. 425). Если у типа элемента нет стандартного конструктора, то наряду с размером следует определить явный инициализатор элемента.



Конструкторы, получающие размер, допустимы *только* для последовательных контейнеров; ассоциативные контейнеры их не поддерживают.

Библиотечные массивы имеют фиксированный размер

Подобно тому, как размер встроенного массива является частью его типа, размер библиотечного контейнера `array` тоже входит в состав его типа. Когда определяется массив, кроме типа элемента задается также размер контейнера:

```
array<int, 42> // тип: массив, содержащий 42 целых числа
array<string, 10> // тип: массив, содержащий 10 строк
```

Чтобы использовать контейнер `array`, следует указать тип элемента и его размер:

```
array<int, 10>::size_type i; // тип массива включает тип элемента
                          // и размер
array<int>::size_type j; // ошибка: array<int> - это не тип
```

Поскольку размер является частью типа массива, контейнер `array` не поддерживает обычные конструкторы контейнерных классов. Эти конструкторы, явно или неявно, определяют размер контейнера. В случае массива разрешение пользователям передавать аргумент размера было бы избыточно (в лучшем случае) и приводило бы к ошибкам.

Фиксированный характер размера массивов влияет также на поведение остальных конструкторов, действительно определяющих массив. В отличие от других контейнеров, созданный по умолчанию массив не пуст: количество его элементов соответствует размеру, а инициализированы они значением по умолчанию (см. раздел 2.2.1, стр. 77), как и элементы встроенного массива (см. раздел 3.5.1, стр. 162). При списочной инициализации массива количество инициализаторов не должно превышать размер массива. Если инициализаторов меньше, чем элементов массива, они используются для первых элементов, а все остальные инициализируются значением по умолчанию (см. раздел 3.3.1, стр. 143). В любом случае, если типом элемента является класс, то у него должен быть стандартный конструктор, обеспечивающий инициализацию значением по умолчанию:

```
array<int, 10> ia1;           // десять целых чисел, инициализированных
                             // значением по умолчанию
array<int, 10> ia2 = {0,1,2,3,4,5,6,7,8,9}; // списочная инициализация
array<int, 10> ia3 = {42}; // ia3[0] содержит значение 42, остальные
                           // элементы - значение 0
```

Следует заметить, что хоть и нельзя копировать или присваивать объекты массивов встроенных типов (см. раздел 3.5.1, стр. 162), для контейнеров `array` такого ограничения нет:

```
int digs[10] = {0,1,2,3,4,5,6,7,8,9};
int spy[10] = digs; // ошибка: встроенные массивы не допускают
                    // копирования и присвоения
array<int, 10> digits = {0,1,2,3,4,5,6,7,8,9};
array<int, 10> copy = digits; // ok: если типы массивов совпадают
```

Как обычно, инициализирующий контейнер должен иметь тот же тип, что и создаваемый. Для контейнера `array` совпадать должны тип элемента и размер, поскольку размер массива — часть его типа.

Упражнения раздела 9.2.4

Упражнение 9.11. Приведите пример каждого из шести способов создания и инициализации контейнеров `vector`. Объясните, какие значения будет содержать каждый вектор.

Упражнение 9.12. Объясните различие между конструктором, получающим контейнер для копирования, и конструктором получающим два итератора.

Упражнение 9.13. Как инициализировать контейнер `vector<double>` из контейнера `list<int>` и контейнера `vector<int>`? Напишите код для проверки ответов.

9.2.5. Присвоение и функция `swap()`

Связанные с присвоением операторы, перечисленные в табл. 9.4, воздействуют на весь контейнер. Оператор присвоения заменяет весь диапазон элементов в левом контейнере копиями элементов из правого:

```
c1 = c2;           // заменяет содержимое контейнера c1 копией
                  // элементов контейнера c2
c1 = {a,b,c};      // после присвоения контейнер c1 имеет размер 3
```

После первого присвоения контейнеры слева и справа равны. Если контейнеры имели неравный размер, после присвоения у обоих будет размер контейнера из правого операнда. После второго присвоения размер контейнера `c1` составит 3, что соответствует количеству значений, представленных в списке.

Таблица 9.4. Операторы присвоения контейнеров

<code>c1 = c2</code>	Заменяет элементы контейнера <code>c1</code> копиями элементов контейнера <code>c2</code> . Контейнеры <code>c1</code> и <code>c2</code> должны иметь тот же тип
<code>c = {a,b,c...}</code>	Заменяет элементы контейнера <code>c1</code> копиями элементов списка инициализации. (Недопустимо для массива.)
<code>swap(c1, c2)</code> <code>c1.swap(c2)</code>	Обменивает элементы контейнеров <code>c1</code> и <code>c2</code> . Контейнеры <code>c1</code> и <code>c2</code> должны иметь тот же тип. Обычно функция <code>swap()</code> выполняется <i>намного быстрее</i> , чем процесс копирования элементов из контейнера <code>c2</code> в <code>c1</code>
Операторы присвоения недопустимы для ассоциативных контейнеров и массива	
<code>seq.assign(b,e)</code>	Заменяет элементы в контейнере <code>seq</code> таковыми из диапазона, обозначенного итераторами <code>b</code> и <code>e</code> . Итераторы <code>b</code> и <code>e</code> не должны ссылаться на элементы в контейнере <code>seq</code>
<code>seq.assign(il)</code>	Заменяет элементы в контейнере <code>seq</code> таковыми из списка инициализации <code>il</code>
<code>seq.assign(n,t)</code>	Заменяет элементы в контейнере <code>seq</code> набором из <code>n</code> элементов со значением <code>t</code>

В отличие от встроенных массивов, библиотечный тип `array` поддерживает присвоение. У левых и правых операндов должен быть одинаковый тип:

```
array<int, 10> a1 = {0,1,2,3,4,5,6,7,8,9};
array<int, 10> a2 = {0}; // все элементы со значением 0
a1 = a2; // замена элементов в a1
a2 = {0}; // ошибка: нельзя присвоить массиву значения из списка
```

Поскольку размер правого операнда может отличаться от размера левого операнда, тип `array` не поддерживает функцию `assign()` и это не позволяет присваивать значения из списка.



Связанные с присвоением операторы делают недопустимыми итераторы, ссылки и указатели на контейнер слева.

Применение функции `assign()` (только последовательные контейнеры)

Оператор присвоения требует совпадения типов левых и правых операндов. Он копирует все элементы правого операнда в левый. Последовательные контейнеры (кроме `array`) определяют также функцию-член `assign()`, обеспечивающую присвоение разных, но совместимых типов, или присвоение контейнерам последовательностей. Функция `assign()` заменяет все элементы в левом контейнере копиями элементов, указанных в ее аргументе. Например, функцию `assign()` можно использовать для присвоения диапазона значений `char*` из вектора в список строк:

```
list<string> names;
vector<const char*> oldstyle;
names = oldstyle; // ошибка: типы контейнеров не совпадают
// ok: преобразование из const char* в string возможно
names.assign(oldstyle.cbegin(), oldstyle.cend());
```

Вызов функции `assign()` заменяет элементы в контейнере `names` копиями элементов из диапазона, обозначенного итераторами. Аргументы функции `assign()` определяют количество элементов контейнера и их значения.



Поскольку существующие элементы заменяются, итераторы, переданные функции `assign()`, не должны относиться к контейнеру, для которого вызвана функция `assign()`.

Вторая версия функции `assign()` получает целочисленное значение и значение элемента. Она заменяет указанное количество элементов контейнера заданным значением:

```
// эквивалент slist1.clear();
// сопровождается slist1.insert(slist1.begin(), 10, "Hiya!");
list<string> slist1(1); // один элемент; пустая строка
slist1.assign(10, "Hiya!"); // десять элементов; со значением "Hiya!"
```

Применение функции `swap()`

Функция `swap()` меняет местами значения двух контейнеров того же типа. Типы контейнеров и их элементов должны совпадать. После обращения к функции `swap()` элементы правого операнда оказываются в левом, и наоборот:

```
vector<string> svec1(10); // вектор из 10 элементов
vector<string> svec2(24); // вектор из 24 элементов
svec1.swap(svec2);
```

После выполнения функции `swap()` вектор `svec1` содержит 24 строки, а вектор `svec2` — 10. За исключением массивов смена двух контейнеров осуществляется очень быстро — сами элементы не меняются; меняются лишь внутренние структуры данных.



За исключением массивов функция `swap()` не копирует, не удаляет и не вставляет элементы, поэтому она гарантированно выполняется за постоянное время.

Благодаря тому факту, что элементы не перемещаются, итераторы, ссылки и указатели в контейнере, за исключением контейнера `string`, остаются допустимыми. Они продолжают указывать на те же элементы, что и перед перестановкой. Однако после вызова функции `swap()` эти элементы находятся в другом контейнере. Предположим, например, что итератор `iter` обозначал в векторе строк позицию `svec1[3]`. После вызова функции `swap()` он обозначит элемент в позиции `svec2[3]`. В отличие от других контейнеров, вызов функции `swap()` для строки делает некорректными итераторы, ссылки и указатели.

В отличие от поведения функции `swap()` с другими контейнерами, когда дело доходит до массивов, элементы действительно обмениваются. Обмен двух массивов потребует времени, пропорционального количеству их элементов.

После выполнения функции `swap()`, указатели, ссылки и итераторы остаются связанными с тем же элементом, который они обозначили ранее. Конечно, значение самого элемента заменено значением соответствующего элемента другого массива.



Новая библиотека предоставляет функцию `swap()` в версии члена класса контейнера и в версии, не являющейся членом какого-либо класса. Прежние версии библиотеки определили только версию функции-члена `swap()`. Функция `swap()` не являющаяся членом класса контейнера, имеет большее значение в обобщенных программах. Как правило, лучше использовать именно эту версию.

Упражнения раздела 9.2.5

Упражнение 9.14. Напишите программу, присваивающую значения элементов списка указателей на символьные строки в стиле C (тип `char*`) элементам вектора строк.



9.2.6. Операции с размером контейнера

За одним исключением у классов контейнеров есть три функции, связанные с размером. Функция-член `size()` (см. раздел 3.2.2, стр. 130) возвращает количество элементов в контейнере; функция-член `empty()` возвращает логическое значение `true`, если контейнер пуст, и значение `false` в противном случае; функция-член `max_size()` возвращает число, большее или равное количеству элементов, которые может содержать контейнер данного типа. По причинам, рассматриваемым в следующем разделе, контейнер `forward_list` предоставляет функции `max_size()` и `empty()`, но не функцию `size()`.

9.2.7. Операторы сравнения

Для сравнения используется тот же реляционный оператор, который определен для типа элементов: при сравнении двух контейнеров на неравенство (`!=`) используется оператор `!=` типа их элементов. Если тип элемента не поддерживает определенный оператор, то для сравнения контейнеров такого типа данный оператор использовать нельзя.

Сравнение двух контейнеров осуществляется на основании сравнения пар их элементов. Эти операторы работают так же, как и таковые у класса `string` (см. раздел 3.2.2, стр. 131):

- Если оба контейнера имеют одинаковый размер и все их элементы совпадают, контейнеры равны, в противном случае — не равны.
- Если контейнеры имеют различный размер, но каждый элемент короткого совпадает с соответствующим элементом длинного, считается, что короткий контейнер меньше длинного.
- Если значения элементов контейнеров не совпадают, результат их сравнения зависит от значений первых неравных элементов.

Проще всего понять работу операторов, рассмотрев их на примерах.

```
vector<int> v1 = { 1, 3, 5, 7, 9, 12 };
vector<int> v2 = { 1, 3, 9 };
vector<int> v3 = { 1, 3, 5, 7 };
vector<int> v4 = { 1, 3, 5, 7, 9, 12 };
v1 < v2    // true; v1 и v2 отличаются элементом [2]: v1[2] меньше,
          // чем v2[2]
v1 < v3    // false; все элементы равны, но у v3 их меньше;
v1 == v4   // true; все элементы равны и размер v1 и v4 одинаков
v1 == v2   // false; v2 имеет меньше элементов, чем v1
```

При сравнении контейнеров используются операторы сравнения их элементов



Сравнить два контейнера можно только тогда, когда используемый оператор сравнения определен для типа элемента контейнера.

Операторы равенства контейнеров используют оператор `==` элемента, а операторы сравнения — оператор `<` элемента. Если тип элемента не предоставляет необходимый оператор, то не получится использовать соответствующие операторы и с содержащими их контейнерами. Например, определенный в главе 7 тип `Sales_data` не предоставлял операторов `==` и `<`. Поэтому нельзя сравнить два контейнера, содержащих элементы типа `Sales_data`:

```
vector<Sales_data> storeA, storeB;  
if (storeA < storeB) // ошибка: Sales_data не имеет оператора <
```

Упражнения раздела 9.2.7

Упражнение 9.15. Напишите программу, выясняющую, равны ли два вектора `vector<int>`.

Упражнение 9.16. Перепишите предыдущую программу, но сравните элементы списка `list<int>` и вектора `vector<int>`.

Упражнение 9.17. Допустим, `c1` и `c2` являются контейнерами. Какие условия налагают типы их элементов в следующем выражении?

```
if (c1 < c2)
```

9.3. Операции с последовательными контейнерами

Последовательные и ассоциативные контейнеры отличаются организацией своих элементов. Это различие влияет на способ хранения, доступа, добавления и удаления элементов. В предыдущем разделе рассматривались операции, общие для всех контейнеров (см. табл. 9.2). В остальной части этой главы рассматриваются операции, специфические для последовательных контейнеров.



9.3.1. Добавление элементов в последовательный контейнер

За исключением массива все библиотечные контейнеры обеспечивают гибкое управление памятью. Они позволяют добавлять и удалять элементы, динамически изменяя размер контейнера во время выполнения. В табл. 9.5 (стр. 440) перечислены функции, позволяющие добавлять элементы в последовательный контейнер (но не в массив).

Используя эти функции, следует помнить, что контейнеры применяют различные стратегии резервирования памяти для элементов и что эти стратегии влияют на производительность. Например, добавление элементов в любое место вектора или строки (но не в конец) либо в любое место двухсторонней очереди (но не в начало или в конец) требует перемещения элементов.

Кроме того, добавление элементов в вектор или строку может привести к повторному резервированию памяти всего объекта. Это, в свою очередь, потребует резервирования новой памяти для элементов и их перемещения из старых областей в новые.

Таблица 9.5. Функции, добавляющие элементы в последовательный контейнер

Эти функции изменяют размер контейнера; они не поддерживаются массивами. Контейнер <code>forward_list</code> обладает специальными версиями функций <code>insert()</code> и <code>emplace()</code>; см. раздел 9.3.4 (стр. 450), а функций <code>push_back()</code> и <code>emplace_back()</code> у него нет. Функции <code>push_front()</code> и <code>emplace_front()</code> недопустимы для контейнеров <code>vector</code> и <code>string</code>.	
<code>c.push_back(t)</code> <code>c.emplace_back(args)</code>	Создает в конце контейнера <code>c</code> элемент со значением <code>t</code> или переданным аргументом <code>args</code> . Возвращает тип <code>void</code>
<code>c.push_front(t)</code> <code>c.emplace_front(args)</code>	Создает в начале контейнера <code>c</code> элемент со значением <code>t</code> или переданным аргументом <code>args</code> . Возвращает тип <code>void</code>
<code>c.insert(p, t)</code> <code>c.emplace(p, args)</code>	Создает элемент со значением <code>t</code> или переданным аргументом <code>args</code> перед элементом, обозначенным итератором <code>p</code> . Возвращает итератор на добавленный элемент
<code>c.insert(p, n, t)</code>	Вставляет <code>n</code> элементов со значением <code>t</code> перед элементом, обозначенным итератором <code>p</code> . Возвращает итератор на первый вставленный элемент; если <code>n</code> — нуль, возвращается итератор <code>p</code>
<code>c.insert(p, b, e)</code>	Вставляет элементы из диапазона, обозначенного итераторами <code>b</code> и <code>e</code> перед элементом, обозначенным итератором <code>p</code> . Итераторы <code>b</code> и <code>e</code> не могут относиться к элементам в контейнере <code>c</code> . Возвращает итератор на первый вставленный элемент; если диапазон пуст, возвращается итератор <code>p</code>
<code>c.insert(p, il)</code>	<code>il</code> — это список значений элементов. Вставляет переданные значения перед элементом, обозначенным итератором <code>p</code> . Возвращает итератор на первый вставленный элемент; если список пуст, возвращается итератор <code>p</code>

Применение функции `push_back()`

В разделе 3.3.2 (стр. 146) упоминалось, что функция `push_back()` добавляет элемент в конец вектора. Кроме контейнеров `array` и `forward_list`, каждый последовательный контейнер (включая `string`) поддерживает функцию `push_back()`.

Цикл следующего примера читает по одной строке за раз в переменную `word`:

```
// читать слова со стандартного устройства ввода и помещать
// их в конец контейнера
string word;
while (cin >> word)
    container.push_back(word);
```

Вызов функции `push_back()` создает новый элемент в конце контейнера `container`, увеличивая его размер на 1. Значением этого элемента будет копия значения переменной `word`. Контейнер может иметь любой тип: `list`, `vector` или `deque`.

Поскольку класс `string` — это только контейнер символов, функцию `push_back()` можно использовать для добавления символов в ее конец:

```
void pluralize(size_t cnt, string &word)
{
    if (cnt > 1)
        word.push_back('s'); // то же, что и word += 's'
}
```

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. ЭЛЕМЕНТЫ КОНТЕЙНЕРА СОДЕРЖАТ КОПИИ ЗНАЧЕНИЙ

Когда объект используется для инициализации контейнера или вставки в него, в контейнер помещается копия значения объекта, а не сам объект. Подобно передаче объекта в не ссылочном параметре (см. раздел 6.2.1, стр. 277), между элементом в контейнере и объектом, значение которого было использовано для его инициализации, нет никакой связи. Последующие изменения элемента в контейнере никак не влияют на исходный объект, и наоборот.

Применение функции `push_front()`

Кроме функции `push_back()`, контейнеры `list`, `forward_list` и `deque` предоставляют аналогичную функцию `push_front()`. Она вставляет новый элемент в начало контейнера:

```
list<int> ilist;
// добавить элементы в начало ilist
for (size_t ix = 0; ix != 4; ++ix)
    ilist.push_front(ix);
```

Этот цикл добавляет элементы 0, 1, 2, 3 в начало списка `ilist`. Каждый элемент вставляется в *новое начало* списка, т.е. когда добавляется 1, она оказывается перед 0, 2 — перед 1 и так далее. Таким образом, добавленные в цикле элементы расположены в обратном порядке. После этого цикла список `ilist` содержит такую последовательность: 3, 2, 1, 0.

Обратите внимание: контейнер `deque`, предоставляющий подобно контейнеру `vector` быстрый произвольный доступ к своим элементам, обладает функци-

ей-членом `push_front()`, а контейнер `vector` — нет. Контейнер `deque` гарантирует постоянное время вставки и удаления элементов в начало и в конец контейнера. Как и у контейнера `vector`, вставка элементов иначе как в начало или в конец контейнера `deque` — потенциально продолжительная операция.



Добавление элементов в контейнеры `vector`, `string` или `deque` способно сделать недействительными все существующие итераторы, ссылки и указатели на их элементы.

Добавление элементов в указанную точку контейнера

Функции `push_back()` и `push_front()` предоставляют весьма удобный способ добавления одиночных элементов в конец или в начало последовательного контейнера. Функция `insert()` обладает более общим характером и позволяет вставлять любое количество элементов в любую указанную позицию контейнера. Ее поддерживают контейнеры `vector`, `deque`, `list` и `string`, а контейнер `forward_list` предоставляет собственные специализированные версии этих функций-членов, которые рассматриваются в разделе 9.3.4 (стр. 450).

Каждая из функций вставки получает в качестве первого аргумента итератор, указывающий позицию помещения элемента (элементов) в контейнере. Он может указывать на любую позицию, включая следующий элемент после конца контейнера. Поскольку итератор может указывать на несуществующий элемент после конца контейнера, а также потому, что полезно иметь способ вставки элементов в начало контейнера, элемент (элементы) вставляются *перед* позицией, обозначенной итератором. Рассмотрим следующий оператор:

```
slist.insert(iter, "Hello!"); // вставить "Hello!" прямо перед iter
```

Он вставляет строку со значением `"Hello"` непосредственно перед элементом, обозначенным итератором `iter`.

Даже при том, что у некоторых контейнеров нет функции `push_front()`, к функции `insert()` это не относится. Функция `insert()` позволяет вставлять элементы в начало контейнера, не заботясь о наличии у контейнера функции `push_front()`:

```
vector<string> svec;
list<string> slist;
// эквивалент вызова slist.push_front("Hello!");
slist.insert(slist.begin(), "Hello!");
// вектор не имеет функции push_front(), но вставка перед begin() возможна
// внимание: вставка возможна везде, но вставка в конец вектора может
// потребовать больше времени
svec.insert(svec.begin(), "Hello!");
```



Контейнеры `vector`, `deque` и `string` допускают вставку в любую позицию, но это может потребовать больше времени.

Вставка диапазона элементов

Следующие аргументы функции `insert()`, расположенные после начального итератора, похожи на аргументы конструкторов контейнеров. Версия, получающая количество элементов и значение, добавляет определенное количество одинаковых элементов перед указанной позицией:

```
svec.insert(svec.end(), 10, "Anna");
```

Этот код вставляет 10 элементов в конец вектора `svec` и инициализирует каждый из них строкой "Anna".

Данная версия функции `insert()` получает пару итераторов или список инициализации для вставки элементов из данного диапазона перед указанной позицией:

```
vector<string> v = {"quasi", "simba", "frollo", "scar"};
// вставить два последних элемента вектора v в начало slist
slist.insert(slist.begin(), v.end() - 2, v.end());
slist.insert(slist.end(), {"these", "words", "will",
                          "go", "at", "the", "end"});
// ошибка времени выполнения: обозначающие копируемый диапазон итераторы
// не должны принадлежать тому же контейнеру, который изменяется
slist.insert(slist.begin(), slist.begin(), slist.end());
```

При передаче пары итераторов они не могут относиться к тому же контейнеру, к которому добавляются элементы.



По новому стандарту версии функции `insert()`, получающие количество или диапазон, возвращают итератор на первый вставленный элемент. (В предыдущих версиях библиотеки эти функции возвращали тип `void`.) Если диапазон пуст, никакие элементы не вставляются, а функция возвращает свой первый параметр.

Применение возвращаемого значения функции `insert()`

Значение, возвращенное функцией `insert()`, можно использовать для многократной вставки элементов в определенной позиции контейнера:

```
list<string> lst;
auto iter = lst.begin();
while (cin >> word)
    iter = lst.insert(iter, word); // то же, что и вызов push_front()
```



Важно понимать, почему именно цикл, подобный этому, эквивалентен вызову функции `push_front()`.

Перед циклом итератор `iter` инициализируется возвращаемым значением функции `lst.begin()`. Первый вызов функции `insert()` получает только что прочитанную строку и помещает ее перед элементом, обозначенным итератором `iter`. Значение, возвращенное функцией `insert()`, является итера-

тором на этот новый элемент. Присвоим этот итератор итератору `iter` и повторим цикл, читая другое слово. Пока есть слова для вставки, каждая итерация цикла `while` вставляет новый элемент перед позицией `iter` и снова присваивает ему позицию недавно вставленного элемента. Этот (новый) элемент является первым. Таким образом, каждая итерация вставляет элемент перед первым элементом в списке.

Применение функций `emplace()`

C++ 11 Новый стандарт вводит три новых функции-члена — `emplace_front()`, `emplace()` и `emplace_back()`, которые создают элементы, а не копируют. Они соответствуют функциям `push_front()`, `insert()` и `push_back()`, позволяющим помещать элемент в начало контейнера, перед указанной позицией или в конец контейнера соответственно.

Когда происходит вызов функции-члена `insert()` или `push()`, им передается объект типа элемента для копирования в контейнер. Когда происходит вызов функции `emplace()`, ее аргументы передаются конструктору типа элемента, который создает элемент непосредственно в области, контролируемой контейнером. Предположим, например, что контейнер `s` содержит элементы типа `Sales_data` (см. раздел 7.1.4, стр. 345):

```
// создает объект класса Sales_data в конце контейнера s
// использует конструктор класса Sales_data с тремя аргументами
s.emplace_back("978-0590353403", 25, 15.99);
// ошибка: нет версии функции push_back(), получающей три аргумента
s.push_back("978-0590353403", 25, 15.99);
// ok: создается временный объект класса Sales_data для передачи
// функции push_back()
s.push_back(Sales_data("978-0590353403", 25, 15.99));
```

Вызов функции `emplace_back()` и второй вызов функции `push_back()` создают новые объекты класса `Sales_data`. При вызове функции `emplace_back()` этот объект создается непосредственно в области, контролируемой контейнером. Вызов функции `push_back()` создает локальный временный объект, который помещается в контейнер.

Аргументы функции `emplace()` зависят от типа элемента, они должны соответствовать конструктору типа элемента:

```
// iter указывает на элемент класса Sales_data контейнера s
s.emplace_back(); // использует стандартный конструктор класса Sales_data
s.emplace(iter, "999-999999999"); // используется Sales_data(string)
// использует конструктор класса Sales_data, получающий ISBN, количество
// и цену
s.emplace_front("978-0590353403", 25, 15.99);
```



Функция `emplace()` создает элементы контейнера. Ее аргументы должны соответствовать конструктору типа элемента.

Упражнения раздела 9.3.1

Упражнение 9.18. Напишите программу чтения последовательности строк со стандартного устройства ввода в контейнер `deque`. Для записи элементов в контейнер `deque` используйте итераторы и цикл.

Упражнение 9.19. Перепишите программу из предыдущего упражнения, чтобы использовался контейнер `list`. Перечислите необходимые изменения.

Упражнение 9.20. Напишите программу, копирующую элементы списка `list<int>` в две двухсторонние очереди, причем нечетные элементы должны копироваться в один контейнер `deque`, а четные в другой.

Упражнение 9.21. Объясните, как цикл на стр. 443, использующий возвращаемое значение функции `insert()` и добавляющий элементы в список, работал бы с вектором вместо списка.

Упражнение 9.22. С учетом того, что `iv` является вектором целых чисел, что не так со следующей программой? Как ее можно исправить?

```
vector<int>::iterator iter = iv.begin(),
                    mid = iv.begin() + iv.size()/2;
while (iter != mid)
    if (*iter == some_val)
        iv.insert(iter, 2 * some_val);
```



9.3.2. Доступ к элементам

В табл. 9.6 приведен список функций, которые можно использовать для доступа к элементам последовательного контейнера. Если в контейнере нет элементов, функции доступа неприменимы.

У каждого последовательного контейнера, включая `array`, есть функция-член `front()`, и у всех, кроме `forward_list`, есть также функция-член `back()`. Эти функции возвращают ссылку на первый и последний элементы соответственно:

```
// перед обращением к значению итератора удостовериться, что
// элемент существует, либо вызвать функции front() и back()
if (!c.empty()) {
    // val и val2 - копии значений первого элемента в контейнере c
    auto val = *c.begin(), val2 = c.front();
    // val3 и val4 - копии значений последнего элемента в контейнере c
    auto last = c.end();
    auto val3 = *(--last); // невозможен декремент итератора forward_list
    auto val4 = c.back();  // не поддерживается forward_list
}
```

Таблица 9.6. Функции доступа к элементам последовательного контейнера

Функция <code>at()</code> и оператор индексирования допустимы только для контейнеров <code>string</code>, <code>vector</code>, <code>deque</code> и <code>array</code>. Функция <code>back()</code> недопустима для контейнера <code>forward_list</code>.	
<code>c.back()</code>	Возвращает ссылку на последний элемент контейнера <code>c</code> , если он не пуст
<code>c.front()</code>	Возвращает ссылку на первый элемент контейнера <code>c</code> , если он не пуст
<code>c[n]</code>	Возвращает ссылку на элемент, индексированный целочисленным беззнаковым значением <code>n</code> . Если <code>n > c.size()</code> , результат непредсказуем
<code>c.at(n)</code>	Возвращает ссылку на элемент по индексу <code>n</code> . Если индекс находится вне диапазона, передает исключение <code>out_of_range</code>

Эта программа получает ссылки на первый и последний элементы контейнера `c` двумя разными способами. Прямой подход — обращение к функциям `front()` и `back()`. Косвенный подход получения ссылки на тот же элемент подразумевает обращение к значению итератора, возвращенного функцией `begin()`, или декремент с последующим обращением к значению итератора, возвращенного функцией `end()`.

В этой программе примечательны два момента: поскольку возвращенный функцией `end()` итератор указывает на элемент, следующий после последнего, для доступа к последнему элементу контейнера применяется декремент полученного итератора. Вторым очень важным моментом является необходимость удостовериться в том, что контейнер `c` не пуст, перед вызовом функций `front()` и `back()` или обращением к значению итераторов, возвращенных функциями `begin()` и `end()`. Если контейнер окажется пустым, все выражения в блоке операторов `if` будут некорректны.



ВНИМАНИЕ

Вызов функций `front()` или `back()` для пустого контейнера, равно как и использование индекса, выходящего за диапазон существующих элементов, является серьезной ошибкой.

Функции-члены, обращающиеся к элементам в контейнере (т.е. функции `front()`, `back()`, `at()` и индексирование), возвращают ссылки. Если контейнер является константным объектом, возвращается ссылка на константу. Если контейнер не константа, возвращается обычная ссылка, которую можно использовать для изменения значения выбранного элемента:

```
if (!c.empty()) {
    c.front() = 42;           // присвоить 42 первому элементу в контейнере c
    auto &v = c.back();      // получить ссылку на последний элемент
    v = 1024;                // изменить элемент в контейнере c
}
```



```

auto v2 = c.back(); // v2 не ссылка; это копия c.back()
v2 = 0;             // это не изменит элемент в контейнере c
}

```

Как обычно, если ключевое слово `auto` применяется для определения переменной, сохраняющей значения, возвращаемые одной из этих функций, и эту переменную предстоит использовать для изменения элемента, то определять эту переменную следует как ссылочный тип.

Индексация и безопасный произвольный доступ

Контейнеры, предоставляющие быстрый произвольный доступ (`string`, `vector`, `deque` и `array`), предоставляют также оператор индексирования (см. раздел 3.3.3, стр. 148). Как уже упоминалось, оператор индексирования получает индекс и возвращает ссылку на элемент в этой позиции контейнера. Индекс не должен выходить за диапазон элементов (т.е. больше или равен 0 и меньше размера контейнера). Допустимость индекса должен обеспечить разработчик; оператор индексирования не проверяет принадлежность индекса диапазону. Использование для индекса значения вне диапазона является серьезной ошибкой, но компилятор ее не обнаружит.

Если необходимо гарантировать допустимость индекса, вместо него можно использовать функцию-член `at()`. Она действует как оператор индексирования, но если индекс недопустим, то она передает исключение `out_of_range` (см. раздел 5.6, стр. 257):

```

vector<string> svec; // пустой вектор
cout << svec[0];     // ошибка времени выполнения: вектор svec
                    // еще не имеет элементов!
cout << svec.at(0);  // передает исключение out_of_range

```

Упражнения раздела 9.3.2

Упражнение 9.23. Какими были бы значения переменных `val2`, `val3` и `val4`, в первой программе данного раздела на стр. 445, если бы функция `c.size()` возвращала значение 1?

Упражнение 9.24. Напишите программу, которая обращается к первому элементу вектора, используя функции `at()`, `front()` и `begin()`, а также оператор индексирования. Проверьте программу на пустом векторе.



9.3.3. Удаление элементов

Подобно тому, как существует несколько способов добавления элементов в контейнер (исключая `array`), существует несколько способов их удаления. Функции удаления перечислены в табл. 9.7.

Таблица 9.7. Функции удаления последовательных контейнеров

Эти функции изменяют размер контейнера; они не поддерживаются массивами. Контейнер `forward_list` обладает специальной версией функции `erase()`; см. раздел 9.3.4 (стр. 450), а функции `pop_back()` у него нет. Функция `pop_front()` недопустима для контейнеров `vector` и `string`.

<code>c.pop_back()</code>	Удаляет последний элемент контейнера <code>c</code> . Результат непредсказуем, если контейнер <code>c</code> пуст. Возвращает <code>void</code>
<code>c.pop_front()</code>	Удаляет первый элемент контейнера <code>c</code> . Результат непредсказуем, если контейнер <code>c</code> пуст. Возвращает <code>void</code>
<code>c.erase(p)</code>	Удаляет элемент, обозначенный итератором <code>p</code> . Возвращает итератор на элемент после удаленного или <i>итератор после конца</i> (off-the-end iterator), если итератор <code>p</code> обозначает последний элемент. Результат непредсказуем, если итератор <code>p</code> указывает на следующий элемент после последнего
<code>c.erase(b, e)</code>	Удаляет диапазон элементов, обозначенных итераторами <code>b</code> и <code>e</code> . Возвращает итератор на элемент после последнего удаленного или после последнего элемента контейнера, если итератор <code>e</code> указывал на последний элемент
<code>c.clear()</code>	Удаляет все элементы контейнера <code>c</code> . Возвращает <code>void</code>



ВНИМАНИЕ

Функции-члены удаления элементов не проверяют свои аргументы. Разработчик должен сам позаботиться о проверке наличия элементов перед их удалением.

Применение функций `pop_front()` и `pop_back()`

Функции `pop_front()` и `pop_back()` удаляют, соответственно, первый и последний элементы контейнера. Векторы и строки функциями `push_front()` и `pop_front()` не обладают. У контейнера `forward_list` также нет функции `pop_back()`. Подобно функциям-членам доступа к элементам, эти функции неприменимы к пустому контейнеру.

Удалив соответствующий элемент, эти функции возвращают тип `void`. Если необходимо извлечь элемент, то следует сохранить его значение перед удалением:

```
while (!ilist.empty()) {
    process(ilist.front()); // действий с текущей вершиной списка ilist
    ilist.pop_front();      // готово; удалить первый элемент
}
```



ВНИМАНИЕ

Удаление элементов в любой позиции, кроме начала или конца двухсторонней очереди, объявляет недействительными все итераторы, ссылки и указатели. В векторе и строке недействительными объявляются итераторы, ссылки и указатели на элементы, расположенные после удаленного элемента.

Удаление элемента в середине контейнера

Функции-члены удаления удаляют элемент (элементы) в указанной позиции контейнера. Можно удалить как отдельный элемент, обозначенный итератором, так и диапазон элементов, отмеченных парой итераторов. Обе формы функции `erase()` возвращают итератор на область *после* последнего удаленного элемента.

В качестве примера рассмотрим следующий цикл, удаляющий нечетные элементы списка:

```
list<int> lst = {0,1,2,3,4,5,6,7,8,9};
auto it = lst.begin();
while (it != lst.end())
    if (*it % 2)           // если элемент является нечетным
        it = lst.erase(it); // удалить его
    else
        ++it;
```

На каждой итерации проверяется нечетность текущего элемента. Если это так, то данный элемент удаляется, а итератор `it` устанавливается на следующий элемент после удаленного. Если элемент `*it` четный, осуществляется приращение итератора `it`, чтобы при следующей итерации он указывал на следующий элемент.

Удаление нескольких элементов

Версия функции `erase()` с парой итераторов позволяет удалить диапазон элементов:

```
// удалить диапазон элементов между двумя итераторами
// возвращает итератор на элемент сразу после последнего удаленного
elem1 = slist.erase(elem1, elem2); // после вызова elem1 == elem2
```

Итератор `elem1` указывает на первый удаляемый элемент, а итератор `elem2` — на следующий после последнего удаляемого.

Чтобы удалить все элементы в контейнере, можно либо вызвать функцию `clear()`, либо перебирать итераторы от возвращаемого функцией `begin()` до `end()` и передавать их функции `erase()`:

```
slist.clear(); // удалить все элементы в контейнере
slist.erase(slist.begin(), slist.end()); // эквивалент
```

Упражнения раздела 9.3.3

Упражнение 9.25. Что будет, если в программе на стр. 449, где удалялся диапазон элементов, итераторы `elem1` и `elem2` равны? Что если итератор `elem2` или оба итератора (`elem1` и `elem2`) являются итератором после конца?

Упражнение 9.26. Используя приведенное ниже определение массива `ia`, скопируйте его содержимое в вектор и в список. Используя версию функции `erase()` для одного итератора, удалите из списка элементы с нечетными значениями, а из вектора — с четными.

```
int ia[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 55, 89 };
```



9.3.4. Специализированные функции контейнера `forward_list`

Чтобы лучше понять, почему у контейнера `forward_list` есть специальные версии функций добавления и удаления элементов, рассмотрим, что происходит при удалении элемента из односвязного списка. Как показано на рис. 9.1, удаление элемента изменяет связи в последовательности. В данном случае удаление элемента `elem3` изменяет связь элемента `elem2`; элемент `elem2` указывал на элемент `elem3`, но после удаления элемента `elem3` элемент `elem2` указывает на элемент `elem4`.



Удаление элемента `elem3` изменяет элемент `elem2`

Рис. 9.1. Специализированные функции контейнера `forward_list`

При добавлении или удалении элемента у элемента перед ним будет другой последователь. Чтобы добавить или удалить элемент, необходимо обратиться к его предшественнику и изменить его ссылку. Однако контейнер `forward_list` — это односвязный список. В односвязном списке нет простого способа доступа к предыдущему элементу. Поэтому функции добавления и удаления элементов в контейнере `forward_list` работают, изменяя элемент *после* указанного. Таким образом, у нас всегда есть доступ к элементам, на которые влияет изменение.

Поскольку эти функции ведут себя отлично от функций других контейнеров, класс `forward_list` не определяет функции-члены `insert()`, `emplace()` и `erase()`. Вместо них используются функции-члены `insert_after()`, `emplace_after` и `erase_after()` (перечислены в табл. 9.8). На рисунке выше, например, для удаления элемента `elem3` использовалась бы функция `erase_after()` для итератора, обозначающего элемент `elem2`. Для реализации этих операций класс `forward_list` определяет также функцию `before_begin()`, которая возвращает итератор *после начала* (off-the-beginning iterator). Этот итератор позволяет добавлять или удалять элементы *после* несуществующего элемента перед первым в списке.

Таблица 9.8. Функции вставки и удаления элементов контейнера `forward_list`

<code>lst.before_begin()</code> <code>lst.cbefore_begin()</code>	Возвращает итератор, обозначающий несуществующий элемент непосредственно перед началом списка. К значению этого итератора обратиться нельзя. Функция <code>cbefore_begin()</code> возвращает итератор <code>const_iterator</code> .
<code>lst.insert_after(p, t)</code> <code>lst.insert_after(p, n, t)</code> <code>lst.insert_after(p, b, e)</code> <code>lst.insert_after(p, il)</code>	Вставляет элемент (элементы) <i>после</i> обозначенного итератором <code>p</code> . <code>t</code> — это объект, <code>n</code> — количество, <code>b</code> и <code>e</code> — обозначающие диапазон итераторы (они не должны принадлежать контейнеру <code>lst</code>), <code>il</code> — список, заключенный в скобки. Возвращает итератор на <i>последний</i> вставленный элемент. Если диапазон пуст, возвращается итератор <code>p</code> . Если <code>p</code> — итератор после конца, результат будет непредсказуемым.
<code>emplace_after(p, args)</code>	Параметр <code>args</code> используется для создания элементов после обозначенного итератором <code>p</code> . Возвращает итератор на новый элемент. Если <code>p</code> — итератор после конца, результат будет непредсказуемым.
<code>lst.erase_after(p)</code> <code>lst.erase_after(b, e)</code>	Удаляет элемент <i>после</i> обозначенного итератором <code>p</code> или диапазоном элементов <i>после</i> обозначенного итератором <code>b</code> и <i>до</i> , но не включая обозначенного итератором <code>e</code> . Возвращает итератор на элемент после удаленного или на элемент после конца контейнера. Если <code>p</code> — итератор после конца или последний элемент контейнера, результат будет непредсказуемым.

При добавлении или удалении элементов в контейнер `forward_list` следует обратить внимание на два итератора: на проверяемый элемент и на элемент, предшествующий ему. В качестве примера перепишем приведенный выше цикл, удалявший из списка нечетные элементы, так, чтобы использовался контейнер `forward_list`:

```
forward_list<int> flst = {0,1,2,3,4,5,6,7,8,9};
auto prev = flst.before_begin(); // обозначает элемент "перед началом"
                                   // контейнера flst
auto curr = flst.begin();        // обозначает первый элемент контейнера flst
while (curr != flst.end()) { // пока есть элементы для обработки
    if (*curr % 2)            // если элемент нечетный
        curr = flst.erase_after(prev); // удалить его и переместить curr
    else {
        prev = curr; // переместить итератор на следующий элемент
        ++curr;      // и один перед следующим элементом
    }
}
```

Здесь итератор `curr` обозначает проверяемый элемент, а итератор `prev` — элемент перед `curr`. Итератор `curr` инициализирует вызов функции `begin()`, чтобы первая итерация проверила на четность первый элемент. Итератор `prev` инициализирует вызов функции `before_begin()`, который возвращает итератор на несуществующий элемент непосредственно перед `curr`.

Когда находится нечетный элемент, итератор `prev` передается функции `erase_after()`. Этот вызов удаляет элемент после обозначенного итератором `prev`; т.е. элемент, обозначенный итератором `curr`. Итератору `curr` присваивается значение, возвращенное функцией `erase_after()`. В результате он обозначит следующий элемент последовательности, а итератор `prev` останется неизменным; он все еще обозначает элемент перед (новым) значением итератора `curr`. Если обозначенный итератором `curr` элемент не является нечетным, то в части `else` оба итератора перемещаются на следующий элемент.

Упражнения раздела 9.3.4

Упражнение 9.27. Напишите программу для поиска и удаления нечетных элементов в контейнере `forward_list<int>`.

Упражнение 9.28. Напишите функцию, получающую контейнер `forward_list<string>` и два дополнительных аргумента типа `string`. Функция должна находить первую строку и вставлять вторую непосредственно после первой. Если первая строка не найдена, то вставьте вторую строку в конец списка.

9.3.5. Изменение размеров контейнера

Для изменения размера контейнера, за исключением массива, можно использовать функцию `resize()`, представленную в табл. 9.9. Если текущий размер больше требуемого, элементы удаляются с конца контейнера; если текущий размер меньше нового, элементы добавляются в конец контейнера:

```
list<int> ilist(10, 42); // 10 целых чисел со значением 42
ilist.resize(15);       // добавляет 5 элементов со значением 0
                        // в конец списка ilist
ilist.resize(25, -1);   // добавляет 10 элементов со значением -1
                        // в конец списка ilist
ilist.resize(5);        // удаляет 20 элементов от конца списка ilist
```

Функция `resize()` получает необязательный аргумент — значение элемента, используемое для инициализации всех добавляемых элементов. Если этот аргумент отсутствует, добавленные элементы инициализируются значением по умолчанию (см. раздел 3.3.1, стр. 143). Если контейнер хранит элементы типа класса и функция `resize()` добавляет элементы, то либо следует предоставить инициализатор, либо тип элемента должен иметь стандартный конструктор.

Таблица 9.9. Функции размера последовательного контейнера

За исключением массива	
<code>c.resize(n)</code>	Измените размеры контейнера <code>c</code> так, чтобы у него было <code>n</code> элементов. Если <code>n < c.size()</code> , то лишние элементы отбрасываются. Если следует добавить новые элементы, они инициализируются значением по умолчанию
<code>c.resize(n, t)</code>	Измените размеры контейнера <code>c</code> так, чтобы у него было <code>n</code> элементов. Все добавляемые элементы получают значение <code>t</code>



Если функция `resize()` сокращает контейнер, то итераторы, ссылки и указатели на удаленные элементы окажутся некорректными; выполнение функции `resize()` для контейнеров `vector`, `string` и `deque` может сделать некорректными все итераторы, указатели и ссылки.

Упражнения раздела 9.3.5

Упражнение 9.29. Если контейнер `vec` содержит 25 элементов, то что делает выражение `vec.resize(100)`? Что если затем последует вызов `vec.resize(10)`?

Упражнение 9.30. Какие ограничения (если они есть) налагает использование функции `resize()` с одиночным аргументом, имеющим тип элемента?



9.3.6. Некоторые операции с контейнерами делают итераторы недопустимыми

Функции, добавляющие и удаляющие элементы из контейнера, могут сделать некорректными указатели, ссылки или итераторы на его элементы. Некорректными считаются те указатели, ссылки и итераторы, которые больше не указывают на элемент. Использование некорректного указателя, ссылки или итератора является серьезной ошибкой, последствия которой, вероятно, будут схожи с использованием неинициализированного указателя (см. раздел 2.3.2, стр. 89).

После операции добавления элементов в контейнер возможно следующее.

- Итераторы, указатели и ссылки на элементы вектора или строки становятся недопустимыми после повторного резервирования пространства контейнера. Если повторного резервирования не было, ссылки на элементы перед позицией вставки остаются допустимыми, а на элементы после позиции вставки — нет.

- Итераторы, указатели и ссылки на элементы двухсторонней очереди становятся недопустимыми после добавления элементов в любую позицию кроме начала или конца. При добавлении в начало или в конец недопустимыми становятся только итераторы, а ссылки и указатели на существующие элементы — нет.

Нет ничего удивительного в том, что после удаления элементов из контейнера итераторы, указатели и ссылки на удаленные элементы становятся недопустимыми. В конце концов, этих элементов больше нет.

- У контейнеров `list` и `forward_list` все остальные итераторы, ссылки и указатели (включая итераторы после конца и перед началом) остаются допустимыми.
- У контейнера `deque` все остальные итераторы, ссылки и указатели становятся недопустимыми, если удалены элементы в любой позиции, кроме начала или конца. Если удаляются элементы в конце, итератор после конца становится недопустимым, но другие итераторы, ссылки и указатели остаются вполне допустимыми. То же относится к удалению из начала.
- У контейнеров `vector` и `string` все остальные итераторы, ссылки и указатели на элементы перед позицией удаления остаются допустимыми. При удалении элементов итератор после конца всегда оказывается недопустимым.

**ВНИМАНИЕ**

Использование недопустимого итератора, указателя или ссылки является серьезной ошибкой, которая проявится во время выполнения программы.

СОВЕТ. КОНТРОЛЬ ИТЕРАТОРОВ

При использовании итератора (или ссылки, или указателя на элемент контейнера) имеет смысл минимизировать ту часть программы, где итератор обязан оставаться допустимым.

Поскольку код, добавляющий или удаляющий элементы из контейнера, может сделать итераторы недопустимыми, необходимо позаботиться о переустановке итераторов соответствующим образом после каждой операции, которая изменяет контейнер. Это особенно важно для контейнеров `vector`, `string` и `deque`.



Создание циклов, которые изменяют контейнер

Циклы, добавляющие или удаляющие элементы из контейнеров `vector`, `string` или `deque`, должны учитывать тот факт, что итераторы, ссылки и указатели могут стать недопустимыми. Программа должна гарантировать, что

итератор, ссылка или указатель обновляется на каждом цикле. Если цикл использует функцию `insert()` или `erase()`, обновить итератор довольно просто. Они возвращают итераторы, которые можно использовать для переустановки итератора:

```
// бесполезный цикл, удаляющий четные элементы и вставляющий дубликаты
// нечетных
vector<int> vi = {0,1,2,3,4,5,6,7,8,9};
auto iter = vi.begin(); // поскольку vi изменяется, используется
                        // функция begin(), а не cbegin()
while (iter != vi.end()) {
    if (*iter % 2) {
        iter = vi.insert(iter, *iter); // дублирует текущий элемент
        iter += 2;                     // переместить через элемент
    } else
        iter = vi.erase(iter);         // удалить четные элементы
        // не перемещать итератор; iter обозначает элемент после
        // удаленного
}
```

Эта программа удаляет четные элементы и дублирует соответствующие нечетные. После вызова функций `insert()` и `erase()` итератор обновляется, поскольку любая из них способна сделать итератор недопустимой.

После вызова функции `erase()` никакой необходимости в приращении итератора нет, поскольку возвращенный ею итератор обозначает следующий элемент в последовательности. После вызова функции `insert()` итератор увеличивается на два. Помните, функция `insert()` осуществляет вставку *перед* указанной позицией и возвращает итератор на вставленный элемент. Таким образом, после вызова функции `insert()` итератор `iter` обозначает элемент (недавно добавленный) перед обрабатываемым. Приращение на два осуществляется для того, чтобы перескочить через добавленный и только что обработанный элементы. Это перемещает итератор на следующий необработанный элемент.



Не храните итератор, возвращенный функцией `end()`

При добавлении или удалении элементов в контейнер `vector` или `string` либо при добавлении или удалении элементов в любую, кроме первой, позицию контейнера `deque` возвращенный функцией `end()` итератор всегда будет недопустимым. Потому циклы, которые добавляют или удаляют элементы, всегда должны вызывать функцию `end()`, а не использовать хранимую копию. Частично поэтому стандартные библиотеки C++ реализуют обычно функцию `end()` так, чтобы она выполнялась очень быстро.

Рассмотрим, например, цикл, который обрабатывает каждый элемент и добавляет новый элемент после исходного. Цикл должен игнорировать добавленные элементы и обрабатывать только исходные. После каждой вставки итератор позиционируется так, чтобы обозначить следующий исходный эле-

мент. Если попытаться “оптимизировать” цикл, сохраняя итератор, возвращенный функцией `end()`, то будет беда:

```
// ошибка: поведение этого цикла непредсказуемо
auto begin = v.begin(),
    end = v.end(); // плохая идея хранить значение итератора end
while (begin != end) {
    // некоторые действия
    // вставить новое значение и переприсвоить итератор begin, который
    // в противном случае окажется недопустимым
    ++begin; // переместить begin, поскольку вставка необходима после
            // этого элемента
    begin = v.insert(begin, 42); // вставить новое значение
    ++begin; // переместить begin за только что добавленный элемент
}
```

Поведение этого кода непредсказуемо. На многих реализациях получится бесконечный цикл. Проблема в том, что возвращенное функцией `end()` значение хранится в локальной переменной `end`. В теле цикла добавляется элемент. Добавление элемента делает итератор, хранимый в переменной `end`, недопустимым. Этот итератор не указывает ни на какой элемент в контейнере `v`, ни на следующий после его конца.



Не кешируйте возвращаемый функцией `end()` итератор в циклах, которые вставляют или удаляют элементы в контейнере `deque`, `string` или `vector`.

Вместо того чтобы хранить итератор `end`, его следует повторно вычислять после каждой вставки:

```
// существенно безопасный: повторно вычислять end после каждого
// добавления/удаления элементов
while (begin != v.end()) {
    // некоторые действия
    ++begin; // переместить begin, поскольку вставка необходима после
            // этого элемента
    begin = v.insert(begin, 42); // вставить новое значение
    ++begin; // переместить begin за только что добавленный элемент
}
```

Упражнения раздела 9.3.6

Упражнение 9.31. Программа на стр. 455, удаляющая четные и дублирующая нечетные элементы, не будет работать с контейнером `list` или `forward_list`. Почему? Переделайте программу так, чтобы она работала и с этими типами тоже.

Упражнение 9.32. Будет ли допустим в программе на стр. 455 следующий вызов функции `insert()`? Если нет, то почему?

```
iter = vi.insert(iter, *iter++);
```

Упражнение 9.33. Что будет, если в последнем примере этого раздела не присваивать переменной `begin` результат вызова функции `insert()`? Напишите программу без этого присвоения, чтобы убедиться в правильности своего предположения.

Упражнение 9.34. Учитывая, что `vi` является контейнером целых чисел, содержащим четные и нечетные значения, предскажите поведение следующего цикла. Проанализировав этот цикл, напишите программу, чтобы проверить правильность своих ожиданий.

```
iter = vi.begin();
while (iter != vi.end())
    if (*iter % 2)
        iter = vi.insert(iter, *iter);
    ++iter;
```



9.4. Как увеличивается размер вектора

Для обеспечения быстрого произвольного доступа элементы вектора хранятся последовательно — каждый элемент рядом с предыдущим. Как правило, нас не должно заботить то, как реализован библиотечный тип; достаточно знать, как его использовать. Но в случае векторов и строк реализация частично просачивается в интерфейс.

С учетом того, что элементы последовательны и размер контейнера гибок, рассмотрим происходящее при добавлении элемента в вектор или строку: если для нового элемента нет места, контейнер не сможет просто добавить элемент в некую другую область памяти — элементы должны располагаться последовательно. Поэтому контейнер должен зарезервировать новую область памяти, достаточную для содержания уже существующих элементов, плюс новый элемент, а затем переместить элементы из старой области в новую, добавить новый элемент и освободить старую область памяти. Если бы вектор осуществлял резервирование и освобождение памяти при каждом добавлении элемента, его работа была бы неприемлемо медленной.

Во избежание дополнительных затрат конструкторы библиотечных контейнеров используют стратегию, сокращающую количество повторных резервирований. При необходимости выделения новой области памяти реализации классов `vector` и `string` обычно резервируют больший объем, чем необходимо в данный момент. Контейнер хранит его в резерве и использует для размещения новых элементов при их добавлении. Поэтому нет никакой необходимости в повторном резервировании места для контейнера при каждом добавлении нового элемента.

Эта стратегия резервирования существенно эффективней таковой при каждой необходимости добавления нового элемента. Фактически ее производи-

тельность настолько высока, что на практике вектор обычно растет эффективней, чем список или двухсторонняя очередь, хотя вектор должен еще перемещать свои элементы при каждом повторном резервировании памяти.

Функции-члены управления емкостью

Типы `vector` и `string` предоставляют описанные в табл. 9.10 функции-члены, позволяющие взаимодействовать с частью реализации, относящейся к резервированию памяти. Функция `capacity()` сообщает количество элементов, которое контейнер может создать прежде, чем ему понадобится занять больший объем памяти. Функция `reserve()` позволяет задать количество резервируемых элементов.

Таблица 9.10. Управление размером контейнера

Функция <code>shrink_to_fit()</code> допустима только для контейнеров <code>vector</code>, <code>string</code> и <code>deque</code>. Функции <code>capacity()</code> и <code>reserve()</code> допустимы только для контейнеров <code>vector</code> и <code>string</code>.	
<code>c.shrink_to_fit()</code>	Запрос на уменьшение емкости в соответствии с размером
<code>c.capacity()</code>	Количество элементов, которое может иметь контейнер с прежде, чем понадобится повторное резервирование
<code>c.reserve(n)</code>	Резервирование места по крайней мере для <code>n</code> элементов



Функция `reserve()` не изменяет количество элементов в контейнере; она влияет только на объем памяти, предварительно резервируемой вектором.

Вызов функции `reserve()` изменяет емкость вектора, только если требуемое пространство превышает текущую емкость. Если требуемый размер больше текущей емкости, функция `reserve()` резервирует по крайней мере столько места, сколько затребовано (или несколько больше).

Если требуемый размер меньше или равен существующей емкости, функция `reserve()` ничего не делает. В частности, вызов функции `reserve()`, при размере меньшем, чем емкость, не приведет к резервированию контейнером памяти. Таким образом, после вызова функции `reserve()` емкость будет больше или равна переданному ей аргументу.

В результате вызов функции `reserve()` никогда не сократит объем контейнера. Точно так же функция-член `resize()` (см. раздел 9.3.5) изменяет только количество элементов контейнера, а не его емкость. Функцию `resize()` нельзя использовать для сокращения объема памяти, которую контейнер держит в резерве.

C++ 11 В новой библиотеке есть функция `shrink_to_fit()`, позволяющая запросить контейнеры `deque`, `vector` или `string` освободить неиспользуемую память. Вызов этой функции означает, что никакой резервной емкости больше не нужно. Однако реализация имеет право проигнорировать этот запрос. Нет никакой гарантии, что вызов функции `shrink_to_fit()` освободит память.

Функции-члены `capacity()` и `size()`

Очень важно понимать различие между *емкостью* (`capacity`) и *размером* (`size`). Размер — это количество элементов, хранящихся в контейнере в настоящий момент, а емкость — это количество элементов, которое контейнер может содержать, не прибегая к следующей операции резервирования памяти.

Следующий код иллюстрирует взаимосвязь размера и емкости:

```
vector<int> ivec;
// размер нулевой; емкость зависит от реализации
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
// присвоить вектору ivec 24 элемента
for (vector<int>::size_type ix = 0; ix != 24; ++ix)
    ivec.push_back(ix);

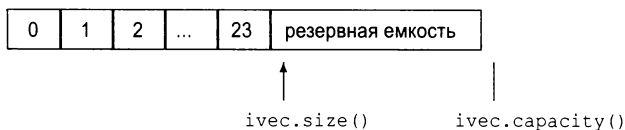
// размер 24; емкость равна или больше 24, согласно реализации
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

При запуске на компьютере автора эта программа отобразила следующий результат:

```
ivec: size: 0 capacity: 0
ivec: size: 24 capacity: 32
```

Как известно, пустой вектор имеет нулевой размер, вполне очевидно, что библиотека для пустого вектора также устанавливает нулевую емкость. При добавлении элементов в вектор его размер составляет количество добавленных элементов. Емкость будет, по крайней мере совпадать с размером, но может быть и больше. Конкретный объем резервной емкости зависит от реализации библиотеки. В данной конкретной реализации добавление 24 элементов по одному приводит к созданию емкости 32.

Визуально текущее состояние вектора `ivec` можно представить так:



Теперь при помощи функции `reserve()` можно зарезервировать дополнительное пространство.

```
ivec.reserve(50); // задать емкость 50 элементов (можно и больше)
// размер будет 24, а емкость - 50 или больше, если так определено
// в реализации
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

Вывод свидетельствует о том, что вызов функции `reserve()` зарезервировал именно столько места, сколько было запрошено:

```
ivec: size: 24 capacity: 50
```

Эту резервную емкость можно впоследствии израсходовать следующим образом:

```
// добавить элементы, чтобы исчерпать резервную емкость
while (ivec.size() != ivec.capacity())
    ivec.push_back(0);
// емкость не изменилась, размер и емкость теперь равны
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

Результат свидетельствует, что в настоящий момент резервная емкость исчерпана, а размер и емкость равны.

```
ivec: size: 50 capacity: 50
```

Поскольку использовалась только резервная емкость, в повторном резервировании нет никакой потребности. Фактически, пока не превышена существующая емкость вектора, никакой необходимости в перераспределении его элементов нет.

Если теперь добавить в вектор новый элемент, то последует повторное резервирование памяти.

```
ivec.push_back(42); // добавить еще один элемент
// размер будет 51, а емкость 51 или больше, если так определено
// в реализации
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

Результат выполнения этой части программы имеет следующий вид:

```
ivec: size: 51 capacity: 100
```

Он свидетельствует о том, что в данной реализации класса `vector` используется стратегия удвоения текущей емкости при каждом резервировании новой области памяти.

По мере необходимости можно вызвать функцию `shrink_to_fit()`, запрашивающую освобождение и возвращение операционной системе памяти, ненужной для текущего размера контейнера:

```
ivec.shrink_to_fit(); // запрос на возвращение памяти
// размер остался неизменным; емкость определена реализацией
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

Вызов функции `shrink_to_fit()` является только запросом; нет никакой гарантии того, что память будет действительно возвращена.



Каждая реализация контейнера `vector` может использовать собственную стратегию резервирования. Однако резервирование новой памяти не должно происходить, пока его емкость не исчерпана.

Вектор может начать повторное резервирование только после выполнения пользователем операции вставки, когда размер равен емкости, вызова функции `resize()` или `reserve()` со значением, превышающим текущую емкость. Количество памяти, резервируемое свыше указанного объема, зависит от реализации.

Каждая реализация обязана следовать стратегии, гарантирующей эффективное использование функции `push_back()` при добавлении элементов в вектор. С технической точки зрения время создания n элементов вектора составляет время выполнения функции `push_back()` для первоначально пустого вектора, умноженное на n .

Упражнения раздела 9.4

Упражнение 9.35. Объясните различие между емкостью вектора и его размером.

Упражнение 9.36. Может ли контейнер иметь емкость, меньшую, чем его размер?

Упражнение 9.37. Почему контейнеры `list` и `array` не имеют функции-члена `capacity()`?

Упражнение 9.38. Напишите программу, позволяющую исследовать рост вектора в библиотеке, которую вы используете.

Упражнение 9.39. Объясните, что выполняет следующий фрагмент программы:

```
vector<string> svec;  
svec.reserve(1024);  
string word;  
while (cin >> word)  
    svec.push_back(word);  
svec.resize(svec.size()+svec.size()/2);
```

Упражнение 9.40. Если программа в предыдущем упражнении читает 256 слов, какова ее вероятная емкость после вызова функции `resize()`? Что, если она читает 512, 1 000 или 1 048 слов?

9.5. Дополнительные операции со строками

Кроме операций, общих для всех последовательных контейнеров, тип `string` предоставляет множество дополнительных. По большей части, эти дополнительные операции либо обеспечивают взаимодействие класса `string` и символьных массивов в стиле C, либо являются дополнительными версиями функций, позволяющими использовать индексы вместо итераторов.

Библиотека `string` определяет множество функций, которые, к счастью, следуют единому шаблону. С учетом количества обсуждаемых функций этот раздел может показаться слишком громоздким при первом чтении; поэтому сначала его можно просто просмотреть. Впоследствии, имея представление о существующих видах операций, можно при необходимости вернуться и изучить подробности использования конкретной функции.



9.5.1. Дополнительные способы создания строк

В дополнение к конструкторам, описанным в разделе 3.2.1 (стр. 126), и конструкторам, общим для всех последовательных контейнеров (см. табл. 9.3, стр. 431), тип `string` предоставляет еще три конструктора, описанные в табл. 9.11.

Таблица 9.11. Дополнительные способы создания строк

Все значения <code>n</code> , <code>len2</code> и <code>pos2</code> являются беззнаковыми.	
<code>string s(cp, n);</code>	Строка <code>s</code> — копия первых <code>n</code> символов из массива, на который указывает <code>cp</code> . У того массива должно быть по крайней мере <code>n</code> символов
<code>string s(s2, pos2);</code>	Строка <code>s</code> — копия символов из строки <code>s2</code> , начиная с позиции по индексу <code>pos2</code> . Если <code>pos2 > s2.size()</code> , результат непредсказуем
<code>string s(s2, pos2, len2);</code>	Строка <code>s</code> — копия <code>len2</code> символов из строки <code>s2</code> , начиная с позиции по индексу <code>pos2</code> . Если <code>pos2 > s2.size()</code> , то результат непредсказуем. Независимо от значения <code>len2</code> , копируется по крайней мере <code>s2.size() - pos2</code> символов

Конструкторы, получающие тип `string` или `const char*`, получают дополнительные (необязательные) аргументы, позволяющие задать количество копируемых символов. При передаче строки можно также указать индекс начала копирования:

```
const char *cp = "Hello World!!!"; // массив с нулевым символом в конце
char noNull[] = {'H', 'i'};        // без нулевого символа в конце
string s1(cp); // копирует cp до нулевого символа; s1 == "Hello World!!!"
```



```

string s2(noNull,2); // копирует два символа из noNull; s2 == "Hi"
string s3(noNull);   // непредсказуемо: noNull не завершается null
string s4(cp + 6, 5); // копирует 5 символов, начиная с cp[6];
                     // s4 == "World"
string s5(s1, 6, 5); // копирует 5 символов, начиная с s1[6];
                     // s5 == "World"
string s6(s1, 6);    // копирует от s1[6] до конца s1; s6 == "World!!!"
string s7(s1,6,20);  // ok, копирует только до конца s1;
                     // s7 == "World!!!"
string s8(s1, 16);   // передает исключение out_of_range

```

Обычно строка создается из типа `const char*`. Массив, на который указывает указатель, должен завершаться нулевым символом; символы копируются до нулевого символа. Если передается также количество, массив не обязан заканчиваться нулевым символом. Если количество не указано и нет нулевого символа или если указанное количество больше размера массива, результат непредсказуем.

При копировании из строки можно предоставить необязательный параметр исходной позиции и количество копируемых символов. Исходная позиция должна быть меньше или равна размеру переданной строки. Если исходная позиция больше размера, то конструктор передаст исключение `out_of_range` (см. раздел 5.6, стр. 257). При передаче количества копирование начинается с указанной позиции. Независимо от количества запрошенных символов, копирование осуществляется до размера строки, но не более.

Функция `substr()`

Функция `substr()` (представленная в табл. 9.12) возвращает копию части или всей исходной строки. Ей можно передать (необязательно) начальную позицию и количество копируемых символов:

```

string s("hello world");
string s2 = s.substr(0, 5); // s2 = hello
string s3 = s.substr(6);   // s3 = world
string s4 = s.substr(6, 11); // s3 = world
string s5 = s.substr(12);  // передает исключение out_of_range

```

Если начальная позиция превышает размер исходной строки, функция `substr()` передает исключение `out_of_range` (см. раздел 5.6, стр. 257). Если начальная позиция в сумме с количеством копируемых символов превосходит размер строки, то копирование осуществляется только до конца строки.

Таблица 9.12. Функция `substr()`

<code>s.substr(pos, n)</code>	Возвращает строку, содержащую <code>n</code> символов из строки <code>s</code> , начиная с позиции <code>pos</code> . По умолчанию параметр <code>pos</code> имеет значение 0. Параметр <code>n</code> по умолчанию имеет значение, подразумевающее копирование всех символов строки <code>s</code> , начиная с позиции <code>pos</code>
-------------------------------	--

Упражнения раздела 9.5.1

Упражнение 9.41. Напишите программу, инициализирующую строку из вектора `vector<char>`.

Упражнение 9.42. Учítывая, что символы в строку следует читать по одному и заранее известно, что прочитать предстоит по крайней мере 100 символов, как можно было бы улучшить производительность программы?



9.5.2. Другие способы изменения строки

Тип `string` поддерживает операторы присвоения последовательного контейнера, а также функции `assign()`, `insert()` и `erase()` (см. раздел 9.2.5 (стр. 435), раздел 9.3.1 (стр. 439) и раздел 9.3.3, (стр. 447)). В нем также определены дополнительные версии функций `insert()` и `erase()`.

В дополнение к обычным версиям функций `insert()` и `erase()`, которые получают итераторы, тип `string` предоставляет их версии, получающие индекс. Индекс указывает начальный элемент для функции `erase()` или начальную позицию для функции `insert()`:

```
s.insert(s.size(), 5, '!'); // вставить пять восклицательных знаков
                          // в конец строки s
s.erase(s.size() - 5, 5);  // удалить последние пять символов из
                          // строки s
```

Библиотека `string` предоставляет также версии функций `insert()` и `assign()`, получающие массивы символов в стиле C. Например, символьный массив с нулевым символом в конце можно использовать как значение, передаваемое функциям `insert()` и `assign()`:

```
const char *cp = "Stately, plump Buck";
s.assign(cp, 7);           // s == "Stately"
s.insert(s.size(), cp + 7); // s == "Stately, plump Buck"
```

Сначала содержимое строки `s` заменяется при вызове функции `assign()`. Присваиваемые строке `s` символы — это семь символов из начала массива, на который указывает указатель `cp`. Количество запрашиваемых символов должно быть меньше или равно количеству символов массива (исключая завершающий нулевой символ), на который указывает `cp`.

Когда происходит вызов функции `insert()` для строки `s`, подразумевается вставка символов перед несуществующим элементом в позиции `s[size()]`. В данном случае копируются символы, начиная с седьмого символа `cp` и до завершающего нулевого символа.

Символы для функций `insert()` и `assign()` можно также указать как исходящие из другой строки или ее подстроки:

```
string s = "some string", s2 = "some other string";
s.insert(0, s2); // вставить копию s2 перед позицией 0 в s
// вставить s2.size() символов из s2, начиная с позиции s2[0] перед s[0]
s.insert(0, s2, 0, s2.size());
```

Функции `append()` и `replace()`

Класс `string` определяет две дополнительные функции-члена, `append()` и `replace()`, способные изменить содержимое строки. Все эти функции описаны в табл. 9.13. Функция `append()` — упрощенный способ вставки в конец:

```
string s("C++ Primer"), s2 = s; // инициализация строк s и s2
// текстом "C++ Primer"
s.insert(s.size(), " 4th Ed."); // s == "C++ Primer 4th Ed."
s2.append(" 4th Ed."); // эквивалент: добавление " 4th Ed." к s2; s == s2
```

Функция `replace()` — упрощенный способ вызова функций `erase()` и `insert()`:

```
// эквивалентный способ замены "4th" на "5th"
s.erase(11, 3); // s == "C++ Primer Ed."
s.insert(11, "5th"); // s == "C++ Primer 5th Ed."
// начиная с позиции 11, удалить три символа, а затем вставить "5th"
s2.replace(11, 3, "5th"); // эквивалент: s == s2
```

В вызове функции `replace()` вставляемый текст может быть того же размера, что и удаляемый. Но можно вставить большую или меньшую строку:

```
s.replace(11, 3, "Fifth"); // s == "C++ Primer Fifth Ed."
```

В этом вызове удаляются три символа, но вместо них вставляются пять.

Таблица 9.13. Функции изменения содержимого строки

<code>s.insert(pos, args)</code>	Вставка символов, определенных аргументом <code>args</code> , перед позицией <code>pos</code> . Позиция <code>pos</code> может быть задана индексом или итератором. Версии, получающие индекс, возвращают ссылку на строку <code>s</code> , а получающие итератор возвращают итератор, обозначающий первый вставленный символ
<code>s.erase(pos, len)</code>	Удаляет <code>len</code> символов, начиная с позиции <code>pos</code> . Если аргумент <code>len</code> пропущен, удаляет символы от позиции <code>pos</code> до конца строки <code>s</code> . Возвращает ссылку на строку <code>s</code>
<code>s.assign(args)</code>	Заменяет символы строки <code>s</code> согласно аргументу <code>args</code> . Возвращает ссылку на строку <code>s</code>
<code>s.append(args)</code>	Добавляет аргумент <code>args</code> к строке <code>s</code> . Возвращает ссылку на строку <code>s</code>
<code>s.replace(range, args)</code>	Удаляет диапазон <code>range</code> символов из строки <code>s</code> и заменяет их символами, заданными аргументом <code>args</code> . Диапазон задан либо индексом и длиной, либо парой итераторов. Возвращает ссылку на строку <code>s</code>

Аргументы <i>args</i> могут быть одним из следующих; функции <code>append()</code> и <code>assign()</code> могут использовать все формы	
Строка <i>str</i> должна быть отлична от <i>s</i>, а итераторы <i>b</i> и <i>e</i> не могут принадлежать строке <i>s</i>	
<i>Str</i>	Строка <i>str</i>
<i>str, pos, len</i>	До <i>len</i> символов из строки <i>str</i> , начиная с позиции <i>pos</i>
<i>cp, len</i>	До <i>len</i> символов из символьного массива, на который указывает указатель <i>cp</i>
<i>Cp</i>	Завершающийся нулевым символом массив, на который указывает указатель <i>cp</i>
<i>n, c</i>	<i>n</i> копий символа <i>c</i>
<i>b, e</i>	Символы в диапазоне, указанном итераторами <i>b</i> и <i>e</i>
Список инициализации	Разделяемый запятыми список символов, заключенный в фигурные скобки

Аргументы <i>args</i> для функций <code>replace()</code> и <code>insert()</code> зависят от того, использован ли диапазон или позиция				
<code>replace (pos, len, args)</code>	<code>replace (b, e, args)</code>	<code>insert (pos, args)</code>	<code>insert (iter, args)</code>	Возможные аргументы <i>args</i>
Да	Да	Да	Нет	<i>str</i>
Да	Нет	Да	Нет	<i>str, pos, len</i>
Да	Да	Да	Нет	<i>cp, len</i>
Да	Да	Нет	Нет	<i>cp</i>
Да	Да	Да	Да	<i>n, c</i>
Нет	Да	Нет	Да	<i>b2, e2</i>
Нет	Да	Нет	Да	список инициализации

Множество способов изменить строку

Функции `append()`, `assign()`, `insert()` и `replace()`, перечисленные в табл. 9.13, имеют несколько перегруженных версий. Аргументы этих функций зависят от того, как заданы добавляемые символы и какая часть строки изменится. К счастью, у этих функций общий интерфейс.

У функций `assign()` и `append()` нет необходимости определять изменяемые части строки: функция `assign()` всегда заменяет все содержимое строки, а функция `append()` всегда добавляет в конец строки.

Функция `replace()` предоставляет два способа определения диапазона удаления символов. Диапазон можно определить по позиции и длине или парой итераторов. Функция `insert()` предоставляет два способа определения позиции вставки: при помощи индекса или итератора. В любом случае новый элемент (элементы) вставляется перед указанным индексом или итератором.

Существует несколько способов определения символов, добавляемых в строку. Новые символы могут быть взяты из другой строки, из указателя на символ, из заключенного в фигурные скобки списка символов или как символ и как число. Когда символы исходят из строки или указателя на символ, можно передать дополнительные аргументы, указывающие, копируются ли все символы аргумента или только часть.

Не каждая функция поддерживает все версии этих аргументов. Например, нет версии функции `insert()`, получающей индекс и список инициализации. Аналогично, если необходимо определить точку вставки, используя итератор, невозможно будет впоследствии передать символьный указатель как источник для новых символов.

Упражнения раздела 9.5.2

Упражнение 9.43. Напишите функцию, получающую три строки: `s`, `oldVal` и `newVal`. Используя итераторы, а также функции `insert()`, и `erase()` замените все присутствующие в строке `s` экземпляры строки `oldVal` строкой `newVal`. Проверьте функцию на примере замены таких общепринятых сокращений, как “tho” на “though” и “thru” на “through”.

Упражнение 9.44. Перепишите предыдущую функцию так, чтобы использовались индекс и функция `replace()`.

Упражнение 9.45. Напишите функцию, получающую строку, представляющую имя и две другие строки, представляющие префикс, такой, как “Mr.” или “Ms.”, а также суффикс, такой, как “Jr.” или “III”. Используя итераторы, а также функции `insert()` и `append()`, создайте новую строку с суффиксом и префиксом, добавленным к имени.

Упражнение 9.46. Перепишите предыдущее упражнение, используя на сей раз позицию, длину и функцию `insert()`.



9.5.3. Операции поиска строк

Класс `string` предоставляет шесть вариантов функций поиска, у каждой из которых есть четыре перегруженных версии. Функции-члены поиска и их аргументы описаны в табл. 9.14. Каждая из них возвращает значение типа `string::size_type`, являющееся индексом найденного элемента. Если соответствие не найдено, функции возвращают статический член (см. раздел 7.6,

стр. 387) по имени `string::npos`. Библиотека определяет значение `npos` как `-1` типа `const string::size_type`. Поскольку `npos` имеет беззнаковый тип, это означает, что значение `npos` соответствует наибольшему возможному размеру, который может иметь любая строка (см. раздел 2.1.2, стр. 66).

Таблица 9.14. Строковые функции поиска

Функции поиска возвращают индекс искомого символа или значение `npos`, если искомое не найдено

<code>s.find(args)</code>	Ищет первое местоположение аргумента <i>args</i> в строке <i>s</i>
<code>s.rfind(args)</code>	Ищет последнее местоположение аргумента <i>args</i> в строке <i>s</i>
<code>s.find_first_of(args)</code>	Ищет первое местоположение любого символа аргумента <i>args</i> в строке <i>s</i>
<code>s.find_last_of(args)</code>	Ищет последнее местоположение любого символа аргумента <i>args</i> в строке <i>s</i>
<code>s.find_first_not_of(args)</code>	Ищет первое местоположение символа в строке <i>s</i> , который отсутствует в аргументе <i>args</i>
<code>s.find_last_not_of(args)</code>	Ищет последнее местоположение символа в строке <i>s</i> , который отсутствует в аргументе <i>args</i>

Аргумент *args* может быть следующим

<i>c</i> , <i>pos</i>	Поиск символа <i>c</i> , начиная с позиции <i>pos</i> в строке <i>s</i> . По умолчанию <i>pos</i> имеет значение 0
<i>s2</i> , <i>pos</i>	Поиск строки <i>s2</i> , начиная с позиции <i>pos</i> в строке <i>s</i> . По умолчанию <i>pos</i> имеет значение 0
<i>cp</i> , <i>pos</i>	Поиск строки с завершающим нулевым символом в стиле C, на которую указывает указатель <i>cp</i> . Поиск начинается с позиции <i>pos</i> в строке <i>s</i> . По умолчанию <i>pos</i> имеет значение 0
<i>cp</i> , <i>pos</i> , <i>n</i>	Поиск первых <i>n</i> символов в массиве, на который указывает указатель <i>cp</i> . Поиск начинается с позиции <i>pos</i> в строке <i>s</i> . Аргумент <i>pos</i> и <i>n</i> не имеют значения по умолчанию



ВНИМАНИЕ

Функции поиска строк возвращают значение беззнакового типа `string::size_type`. Поэтому не следует использовать переменную типа `int` или другого знакового типа для содержания значения, возвращаемого этими функциями (см. раздел 2.1.2, стр. 67).

Самой простой является функция `find()`. Она ищет первое местоположение переданного аргумента и возвращает его индекс или значение `npos`, если соответствующее значение не найдено:

```
string name("AnnaBelle");
auto pos1 = name.find("Anna"); // pos1 == 0
```

Возвращает значение 0, т.е. индекс, по которому подстрока "Anna" расположена в строке "AnnaBelle".

Поиск (и другие операции со строками) чувствительны к регистру. При поиске в строке регистр имеет значение:

```
string lowercase("annabelle");
pos1 = lowercase.find("Anna"); // pos1 == npos
```

Этот код присвоит переменной `pos1` значение `npos`, поскольку строка "Anna" не соответствует строке "anna".

Немного сложнее искать соответствие любому символу в строке. Например, следующий код находит первую цифру в переменной `name`:

```
string numbers("0123456789"), name("r2d2");
// возвращает 1, т.е. индекс первой цифры в имени
auto pos = name.find_first_of(numbers);
```

Кроме поиска соответствия, вызвав функцию `find_first_not_of()`, можно искать первую позицию, которая *не соответствует* искомому аргументу. Например, для поиска первого нечислового символа в строке можно использовать следующий код:

```
string dept("03714p3");
// возвращает 5 - индекс символа 'p'
auto pos = dept.find_first_not_of(numbers);
```

Откуда начинать поиск

Функциям поиска можно передать необязательный аргумент исходной позиции. Этот необязательный аргумент указывает позицию, с которой начинается поиск. По умолчанию значением этого аргумента является нуль. Общепринятой практикой программирования является использование этого аргумента в цикле перебора строки при поиске всех местоположений искомого значения.

```
string::size_type pos = 0;
// каждая итерация находит следующее число в имени
while ((pos = name.find_first_of(numbers, pos))
    != string::npos) {
    cout << "found number at index: " << pos
        << " element is " << name[pos] << endl;
    ++pos; // перевести на следующий символ
}
```

Условие цикла `while` присваивает переменной `pos` индекс первой встретившейся цифры, начиная с текущей позиции `pos`. Пока функция `find_first_of()` возвращает допустимый индекс, результат отображается, а значение `pos` увеличивается.

Если не увеличивать значение переменной `pos` в конце этого цикла, он никогда не завершится, поскольку при последующих итерациях поиск начнется сначала и найден будет тот же элемент. Поскольку значение `pos` так и не будет возвращено, цикл никогда не завершится.

Поиск в обратном направлении

Использованные до сих пор функции поиска выполняется слева направо (т.е. от начала к концу). Библиотека предоставляет аналогичный набор функций, которые просматривают строку справа налево (т.е. от конца к началу). Функция-член `rfind()` ищет последнюю, т.е. расположенную справа, позицию искомой подстроки.

```
string river("Mississippi");  
auto first_pos = river.find("is"); // возвращает 1  
auto last_pos = river.rfind("is"); // возвращает 4
```

Функция `find()` возвращает индекс 1, указывая, что подстрока "is" первый раз встречается, начиная с позиции 1, а функция `rfind()` возвращает индекс 4, указывая начало последнего местонахождения подстроки "is".

Функция `find_last()` аналогична функции `find_first()`, но возвращает *последнее* местоположение, а не первое.

- Функция `find_last_of()` ищет последний символ, который соответствует любому элементу искомой строки.
- Функция `find_last_not_of()` ищет последний символ, который не соответствует ни одному элементу искомой строки.

Каждая из этих функций имеет второй необязательный аргумент, который указывает позицию начала поиска.

Упражнения раздела 9.5.3

Упражнение 9.47. Напишите программу, которая находит в строке "ab2c3d7R4E6" каждую цифру, а затем каждую букву. Напишите две версии программы: с использованием функции `find_first_of()` и функции `find_first_not_of()`.

Упражнение 9.48. С учетом определения переменных `name` и `numbers` на стр. 469, что возвращает вызов функции `numbers.find(name)`?

Упражнение 9.49. У символов может быть надстрочная часть, расположенная выше середины строки, как у `d` или `f`, или подстрочная, ниже середины строки, как у `p` или `g`. Напишите программу, которая читает содержащий слова файл и сообщает самое длинное слово, не содержащее ни надстрочных, ни подстрочных элементов.



9.5.4. Сравнение строк

Кроме операторов сравнения (см. раздел 3.2.2, стр. 131), библиотека `string` предоставляет набор функций сравнения, подобных функции `strcmp()` библиотеки C (см. раздел 3.5.4, стр. 172). Подобно функции `strcmp()`, функция `s.compare()` возвращает нуль, положительное или отрицательное значение, в зависимости от того, равна ли строка `s`, больше или меньше строки, переданной ее аргументом.

Как показано в таб. 9.15, существует шесть версий функции `compare()`. Ее аргументы зависят от того, сравниваются ли две строки или строка и символьный массив. В обоих случаях сравнивать можно либо всю строку, либо ее часть.

Таблица 9.15. Возможные аргументы функции `s.compare()`

<code>s2</code>	Сравнивает строку <code>s</code> со строкой <code>s2</code>
<code>pos1, n1, s2</code>	Сравнивает <code>n1</code> символов, начиная с позиции <code>pos1</code> из строки <code>s</code> , со строкой <code>s2</code>
<code>pos1, n1, s2, pos2, n2</code>	Сравнивает <code>n1</code> символов, начиная с позиции <code>pos1</code> из строки <code>s</code> , со строкой <code>s2</code> , начиная с позиции <code>pos2</code> в строке <code>s2</code>
<code>cp</code>	Сравнивает строку <code>s</code> с завершаемым нулевым символом массивом, на который указывает указатель <code>cp</code>
<code>pos1, n1, cp</code>	Сравнивает <code>n1</code> символов, начиная с позиции <code>pos1</code> из строки <code>s</code> , со строкой <code>cp</code>
<code>pos1, n1, cp, n2</code>	Сравнивает <code>n1</code> символов, начиная с позиции <code>pos1</code> из строки <code>s</code> , со строкой <code>cp</code> , начиная с символа <code>n2</code>



9.5.5. Числовые преобразования

Строки зачастую содержат символы, которые представляют числа. Например, числовое значение 15 можно представить как строку с двумя символами, `'1'` и `'5'`. На самом деле символьное представление числа отличается от его числового значения. Числовое значение 15, хранимое в 16-разрядной переменной типа `short`, будет иметь двоичное значение `00000000000001111`, а символьная строка `"15"`, представленная как два символа из набора `Latin-1`, будет иметь двоичное значение `0011000100110101`. Первый байт представляет

символ '1', восьмеричное значение которого составит 061, а второй байт, представляющий символ '5', в наборе Latin-1 имеет восьмеричное значение 065.



Новый стандарт вводит несколько функций, осуществляющих преобразование между числовыми данными и библиотечным типом `string`.

Таблица 9.16. Преобразования между строками и числами

<code>to_string(val);</code>	Перегруженные версии функции возвращают строковое представление значения <code>val</code> . Аргумент <code>val</code> может иметь любой арифметический тип (см. раздел 2.1.1, стр. 62). Есть версии функции <code>to_string()</code> для любого типа с плавающей точкой и целочисленного типа, включая тип <code>int</code> и большие типы. Малые целочисленные типы преобразуются, как обычно (см. раздел 4.11.1, стр. 219)
<code>stoi(s, p, b)</code> <code>stol(s, p, b)</code> <code>stoul(s, p, b)</code> <code>stoll(s, p, b)</code> <code>stoull(s, p, b)</code>	Возвращают числовое содержимое исходной подстроки <code>s</code> как тип <code>int</code> , <code>long</code> , <code>unsigned long</code> , <code>long long</code> или <code>unsigned long long</code> соответственно. Аргумент <code>b</code> задает используемое для преобразования основание числа; по умолчанию принято значение 10. Аргумент <code>p</code> — указатель на тип <code>size_t</code> , означающий индекс первого нечислового символа в строке <code>s</code> ; по умолчанию <code>p</code> имеет значение 0. В этом случае функция не хранит индекс
<code>stof(s, p)</code> <code>stod(s, p)</code> <code>stold(s, p)</code>	Возвращают числовое содержимое исходной подстроки <code>s</code> как тип <code>float</code> , <code>double</code> или <code>long double</code> соответственно. Аргумент <code>p</code> имеет то же назначение, что и у целочисленных преобразований

```
int i = 42;
string s = to_string(i); // преобразует переменную i типа int в ее
                        // символьное представление
double d = stod(s);     // преобразует строку s в значение типа double
```

Здесь для преобразования числа 42 в его строковое представление используется вызов функции `to_string()`, а затем вызов функции `stod()` преобразует эту строку в значение с плавающей точкой.

Первый преобразуемый в числовое значение символ строки должен быть цифрой:

```
string s2 = "pi = 3.14";
// d = 3.14 преобразуется первая подстрока в строке s, начинающаяся
// с цифры; d = 3.14
d = stod(s2.substr(s2.find_first_of("+-.0123456789")));
```

Для получения позиции первого символа строки `s`, который мог быть частью числа, в этом вызове функции `stod()` используется функция `find_first_of()` (см. раздел 9.5.3, стр. 467). Функции `stod()` передается подстрока строки `s`, начиная с этой позиции. Функция `stod()` читает переданную строку до тех пор, пока не встретится символ, который не может быть частью числа. Затем найденное символьное представление числа преобразуется в соответствующее значение типа `double`.

Первый преобразуемый символ строки должен быть знаком + или – либо цифрой. Строка может начинаться с части 0x или 0X, означающей шестнадцатеричный формат. У функций преобразования чисел с плавающей точкой строка может также начинаться с десятичной точки (.) и содержать символ e или E, означающий экспоненциальную часть. У функций преобразования в целочисленный тип, в зависимости от основания, строка может содержать алфавитные символы, соответствующие цифрам после цифр 9.



Если строка не может быть преобразована в число, эти функции передают исключение `invalid_argument` (см. раздел 5.6, стр. 257). Если преобразование создает значение, которое не может быть представлено заданным типом, они передают исключение `out_of_range`.

Упражнения раздела 9.5.5

Упражнение 9.50. Напишите программу обработки вектора `vector<string>`, элементы которого представляют целочисленные значения. Вычислите сумму всех элементов вектора. Измените программу так, чтобы она суммировала строки, которые представляют значения с плавающей точкой.

Упражнение 9.51. Напишите класс, у которого есть три беззнаковых члена, представляющих год, месяц и день. Напишите конструктор, получающий строку, представляющую дату. Конструктор должен понимать множество форматов даты, такие как January 1, 1900, 1/1/1900, Jan 1, 1900 и т.д.



9.6. Адаптеры контейнеров

Кроме последовательных контейнеров, библиотека предоставляет три адаптера последовательного контейнера: `stack` (стек), `queue` (очередь) и `priority_queue` (приоритетная очередь). *Адаптер* (*adaptor*¹) — это фундаментальная концепция библиотеки. Существуют адаптеры контейнера, итератора и функции. По существу, адаптер — это механизм, заставляющий нечто одно действовать как нечто другое. Адаптер контейнера получает контейнер существующего типа и заставляет его действовать как другой. Например, адаптер `stack` получает любой из последовательных контейнеров (`array` и `forward_list`) и заставляет его работать подобно стеку. Функции и типы данных, общие для всех адаптеров контейнеров, перечислены в табл. 9.17.

¹ Здесь и везде в оригинале именно *adaptor*, а не *adapter*. — Примеч. ред.

Таблица 9.17. Функции и типы, общие для всех адаптеров

<code>size_type</code>	Тип данных, достаточно большой, чтобы содержать размер самого большого объекта этого типа
<code>value_type</code>	Тип элемента
<code>container_type</code>	Тип контейнера, на базе которого реализован адаптер
<code>A a;</code>	Создает новый пустой адаптер по имени <code>a</code>
<code>A a(c);</code>	Создает новый адаптер по имени <code>a</code> , содержащий копию контейнера <code>c</code>
<i>операторы сравнения</i>	Каждый адаптер поддерживает все операторы сравнения: <code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> и <code>>=</code> . Эти операторы возвращают результат сравнения внутренних контейнеров
<code>a.empty()</code>	Возвращает значение <code>true</code> , если адаптер <code>a</code> пуст, и значение <code>false</code> в противном случае
<code>a.size()</code>	Возвращает количество элементов в адаптере <code>a</code>
<code>swap(a, b)</code> <code>a.swap(b)</code>	Меняет содержимое контейнеров <code>a</code> и <code>b</code> ; у них должен быть одинаковый тип, включая тип контейнера, на основании которого они реализованы

Определение адаптера

Каждый адаптер определяет два конструктора: стандартный конструктор, создающий пустой объект, и конструктор, получающий контейнер и инициализирующий адаптер, копируя полученный контейнер. Предположим, например, что `deque` — это двухсторонняя очередь `deque<int>`. Ее можно использовать для инициализации нового стека следующим образом:

```
stack<int> stk(deq); // копирует элементы из deq в stk
```

По умолчанию оба адаптера, `stack` и `queue`, реализованы на основании контейнера `deque`, а адаптер `priority_queue` реализован на базе контейнера `vector`. Заданный по умолчанию тип контейнера можно переопределить, указав последовательный контейнер как второй аргумент при создании адаптера:

```
// пустой стек, реализованный поверх вектора
stack<string, vector<string>> str_stk;
// str_stk2 реализован поверх вектора и первоначально содержит копию svec
stack<string, vector<string>> str_stk2(svec);
```

Существуют некоторые ограничения на применение контейнеров с определенными адаптерами. Всем адаптерам нужна возможность добавлять и удалять элементы. В результате они не могут быть основаны на массиве. Точно

так же нельзя использовать контейнер `forward_list`, поскольку все адаптеры требуют функций добавления, удаления и обращения к последнему элементу контейнера. Стек требует только функций `push_back()`, `pop_back()` и `back()`, поэтому для стека можно использовать контейнер любого из остальных типов. Адаптеру `queue` требуются функции `back()`, `push_back()`, `front()` и `push_front()`, поэтому он может быть создан на основании контейнеров `list` и `deque`, но не `vector`. Адаптеру `priority_queue` в дополнение к функциям `front()`, `push_back()` и `pop_back()` требуется произвольный доступ к элементам; он может быть основан на контейнерах `vector` и `deque`, но не `list`.

Адаптер `stack`

Тип `stack` определен в заголовке `stack`. Функции-члены класса `stack` перечислены в табл. 9.18. Следующая программа иллюстрирует использование адаптера `stack`:

```
stack<int> intStack; // пустой стек
// заполнить стек
for (size_t ix = 0; ix != 10; ++ix)
    intStack.push(ix); // intStack содержит значения 0...9
while (!intStack.empty()) { // пока в intStack есть значения
    int value = intStack.top();
    // код, использующий значение
    intStack.pop(); // извлечь верхний элемент и повторить
}
```

Сначала `intStack` объявляется как пустой стек целочисленных элементов:

```
stack<int> intStack; // пустой стек
```

Затем цикл `for` добавляет десять элементов, инициализируя каждый следующим целым числом, начиная с нуля. Цикл `while` перебирает весь стек, извлекая его верхний элемент, пока он не опустеет.

Таблица 9.18. Функции, поддерживаемые адаптером контейнера `stack`, кроме приведенных в табл. 9.17

По умолчанию используется контейнер <code>deque</code>, но может быть также реализован на основании контейнеров <code>list</code> или <code>vector</code>.	
<code>s.pop()</code>	Удаляет, но не возвращает верхний элемент из стека
<code>s.push(item)</code> <code>s.emplace(args)</code>	Создает в стеке новый верхний элемент, копируя или перемещая элемент <code>item</code> либо создавая элемент из аргумента параметра <code>args</code>
<code>s.top()</code>	Возвращает, но не удаляет верхний элемент из стека

Каждый адаптер контейнера определяет собственные функции, исходя из функций, предоставленных базовым контейнером. Использовать можно только функции адаптера, а функции основного контейнера использовать нельзя. Рассмотрим, например, вызов функции `push_back()` контейнера `deque`, на котором основан стек `intStack`:

```
intStack.push(ix); // intStack содержит значения 0...9
```

Хотя стек реализован на основании контейнера `deque`, прямого доступа к его функциям нет. Для стека нельзя вызвать функцию `push_back()`; вместо нее следует использовать функцию `push()`.

Адаптеры очередей

Адаптеры `queue` и `priority_queue` определены в заголовке `queue`. Список функций, поддерживаемых этими типами, приведен в табл. 9.19.

Таблица 9.19. Функции адаптеров `queue` и `priority_queue`, кроме приведенных в табл. 9.17

По умолчанию адаптер <code>queue</code> использует контейнер <code>deque</code>, а адаптер <code>priority_queue</code> — контейнер <code>vector</code>; адаптер <code>queue</code> может использовать также контейнер <code>list</code> или <code>vector</code>, адаптер <code>priority_queue</code> может использовать контейнер <code>deque</code>.	
<code>q.pop()</code>	Удаляет, но не возвращает первый или наиболее приоритетный элемент из очереди или приоритетной очереди соответственно
<code>q.front()</code> <code>q.back()</code>	Возвращает, но не удаляет первый или последний элемент очереди <code>q</code> . <i>Допустимо только для адаптера <code>queue</code></i>
<code>q.top()</code>	Возвращает, но не удаляет элемент с самым высоким приоритетом. <i>Допустимо только для адаптера <code>priority_queue</code></i>
<code>q.push(item)</code> <code>q.emplace(args)</code>	Создает элемент со значением <code>item</code> или создает его исходя из аргумента <code>args</code> в конце очереди или в соответствующей позиции приоритетной очереди

Библиотечный класс `queue` использует хранилище, организованное по принципу “первым пришел, первым вышел” (first-in, first-out — FIFO). Поступающие в очередь объекты помещаются в ее конец, а извлекаются из ее начала.

Адаптер `priority_queue` позволяет установить приоритет хранимых элементов. Добавляемые элементы помещаются перед элементами с более низким приоритетом. По умолчанию для определения относительных приоритетов в библиотеке используется оператор `<`. Его переопределение рассматривается в разделе 11.2.2 (стр. 543).

Упражнения раздела 9.6

Упражнение 9.52. Используйте стек для обработки выражений со скобками. Встретив открывающую скобку, запомните ее положение. Встретив закрывающую скобку, после открывающей скобки, извлеките эти элементы, включая открывающую скобку, и поместите полученное значение в стек, переместив таким образом заключенное в скобки выражение.

Резюме

Библиотечные типы контейнеров — это шаблоны, позволяющие содержать объекты указанного типа. В последовательных контейнерах элементы упорядочены и доступны по позиции. У последовательных контейнеров общий, стандартизированный интерфейс: если два последовательных контейнера предоставляют некую функцию, то у нее будет тот же интерфейс и значение в обоих контейнерах.

Все контейнеры (кроме контейнера `array`) обеспечивают эффективное управление динамической памятью. Можно добавлять элементы в контейнер, не волнуясь о том, где хранить элементы. Контейнер сам управляет хранением. Контейнеры `vector` и `string` обеспечивают более подробное управление памятью, предоставляя функции-члены `reserve()` и `capacity()`.

По большей части, контейнеры определяют удивительно мало функций. Они определяют конструкторы, функции добавления и удаления элементов, функции выяснения размера контейнера и возвращения итераторов на те или иные элементы. Другие весьма полезные функции, такие как сортировка и поиск, определены не типами контейнеров, а стандартными алгоритмами, которые будут описаны в главе 10.

Функции контейнеров, которые добавляют или удаляют элементы, способны сделать существующие итераторы, указатели или ссылки некорректными. Большинство функций, которые способны сделать итераторы недопустимыми, например `insert()` или `erase()`, возвращают новый итератор, позволяющий не потерять позицию в контейнере. Особую осторожность следует соблюдать в циклах, которые используют итераторы и операции с контейнерами, способные изменить их размер.

Термины

Адаптер `priority_queue` (приоритетная очередь). Адаптер последовательных контейнеров, позволяющий создать очередь, в которой элементы добавляются не в конец, а согласно определенному уровню приоритета. По умолчанию при определении приоритета используется оператор “меньше” для типа элемента.

Адаптер `queue` (очередь). Адаптер последовательных контейнеров, позволяющий создать очередь, в которой элементы добавляются в конец, а предоставляются и удаляются в начале.

Адаптер `stack` (стек). Адаптер последовательных контейнеров, позволяющий создать стек, в который элементы добавляют и удаляют только с одного конца.

Адаптер (`adaptor`). Библиотечный тип, функция или итератор, который заставляет один объект действовать подобно другому. Для последовательных контейнеров существуют три адаптера: `stack`, `queue` и `priority_queue`, каждый из которых определяет новый интерфейс базового последовательного контейнера.

Диапазон итераторов (`iterator range`). Диапазон элементов, обозначенный двумя итераторами. Первый итератор относится к первому элементу в последовательности, а второй — к следующему элементу после последнего. Если диапазон пуст, итераторы равны (и наоборот, если итераторы равны, они обозначают пустой диапазон). Если диапазон не пуст, второй итератор можно достичь последовательным увеличением первого итератора. Последовательное приращение итератора позволяет обработать каждый элемент диапазона.

Интервал, включающий левый элемент (`left-inclusive interval`). Диапазон значений, включающий первый элемент, но исключая последний. Обычно обозначается как `[i, j)`, т.е. начальное значение последовательности `i` включено, а последнее, `j`, исключено.

Итератор после конца (`off-the-end iterator`). Итератор, обозначающий (несуществующий) следующий элемент после последнего в диапазоне. Обычно называемый “конечным итератором” (`end iterator`).

Итератор после начала (`off-the-beginning iterator`). Итератор, обозначающий (несуществующий) элемент непосредственно перед началом контейнера `forward_list`. Возвращается функцией `before_begin()` контейнера `forward_list`. Подобно итератору, возвращенному функцией `end()`, к значению данного итератора обратиться нельзя.

Контейнер `array` (массив). Последовательный контейнер фиксированного размера. Чтобы определить массив, кроме определения типа элемента следует указать его размер. К элементам массива можно обратиться по их позиционному индексу. Обеспечивает быстрый произвольный доступ к элементам.

Контейнер `deque` (двухсторонняя очередь). Последовательный контейнер, к элементам которого можно обратиться по индексу (позиции). Двухсторонняя очередь подобна вектору во всех отношениях, за исключением того, что он обеспечивает быструю вставку как в начало, так и в конец контейнера, без перемещения элементов.

Контейнер `forward_list` (односвязный список). Последовательный контейнер, представляющий односвязный список. К элементам контейнера `forward_list` можно обращаться только последовательно; начиная с данного элемента, можно добраться до другого элемента, только перебрав каждый элемент между ними. Итераторы класса `forward_list` не поддерживают декремент (`--`). Обеспечивает быструю вставку (или удаление) в любой позиции. В отличие от других контейнеров, вставка и удаление происходят *после* указанной позиции итератора. Как следствие, у контейнера `forward_list` есть итератор перед началом, для согласованности с обычным итератором после конца. При добавлении новых элементов итераторы остаются допусти-

мыми. Когда элемент удаляется, некорректным становится лишь итератор удаленного элемента.

Контейнер `list` (список). Последовательный контейнер, к элементам которого можно обратиться только последовательно, т.е. начиная с одного элемента, можно перейти к другому, увеличивая или уменьшая итератор. Итераторы контейнера `list` поддерживают как инкремент (`++`), так и декремент (`-`). Обеспечивает быструю вставку (и удаление) в любой позиции. Добавление новых элементов никак не влияет ни на другие элементы, ни на существующие итераторы. Когда элемент удаляется, некорректным становится лишь итератор удаленного элемента.

Контейнер `vector` (вектор). Последовательный контейнер, к элементам которого можно обратиться по индексу (позиции). Эффективно добавить или удалить элементы вектора можно только с конца. Добавление элементов в вектор может привести к его перераспределению в памяти, что делает некорректными все созданные ранее итераторы. При добавлении (или удалении) элемента в середину вектора итераторы всех расположенных далее элементов становятся некорректными.

Контейнер `container`. Тип (класс), который содержит коллекцию объектов определенного типа. Каждый библиотечный контейнер является шаблоном класса. Чтобы создать контейнер, необходимо указать тип хранимых в нем элементов. За исключением массивов, библиотечные контейнеры имеют переменный размер.

Последовательный контейнер (`sequential container`). Контейнер, позволяющий содержать упорядоченную коллекцию объектов одинакового типа. К элементам последовательного контейнера обращаются по позиции.

Функция `begin()`. Функция контейнера, возвращающая итератор на первый элемент в контейнере (если он есть), или итератор после конца, если контейнер пуст. Будет ли возвращенный итератор константным, зависит от типа контейнера.

Функция `cbegin()`. Функция контейнера, возвращающая итератор `const_iterator` на первый элемент в контейнере (если он есть), или итератор после конца (`off-the-end iterator`), если контейнер пуст.

Функция `cend()`. Функция контейнера, возвращающая итератор `const_iterator` на (несуществующий) элемент после последнего элемента контейнера.

Функция `end()`. Функция контейнера, возвращающая итератор на (несуществующий) элемент после последнего элемента контейнера. Будет ли возвращенный итератор константным, зависит от типа контейнера.

